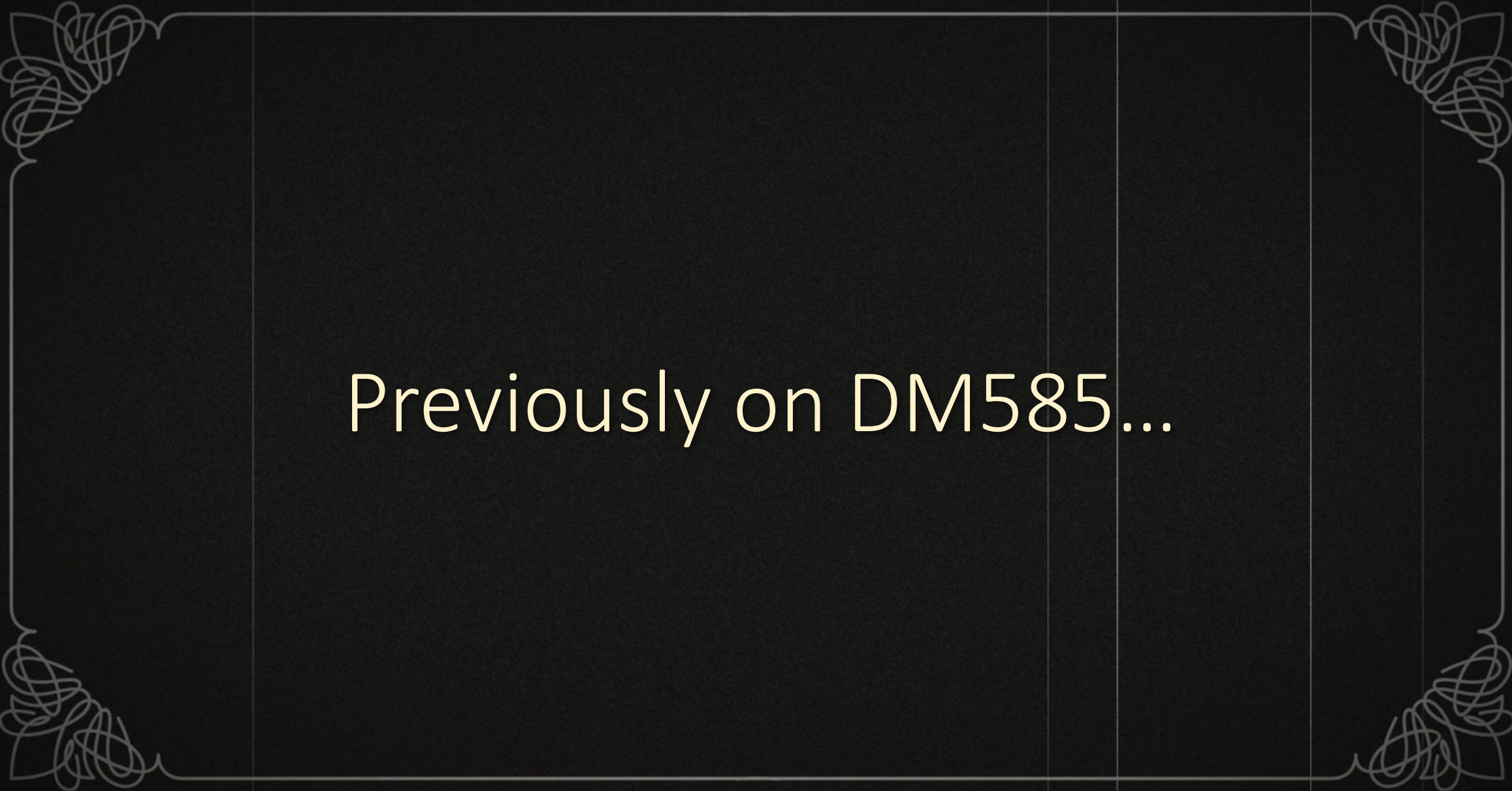


B.1.2 Web Foundations

DM585 Distributed and Web Programming



Previously on DM585...

What is HTML?

HTML (HyperText Markup Language)

- Describes the content and structure of a web page; not a programming language.
- Made up of building blocks called elements.

`<p>`

HTML is `<em>awesome!!!</em>`

``

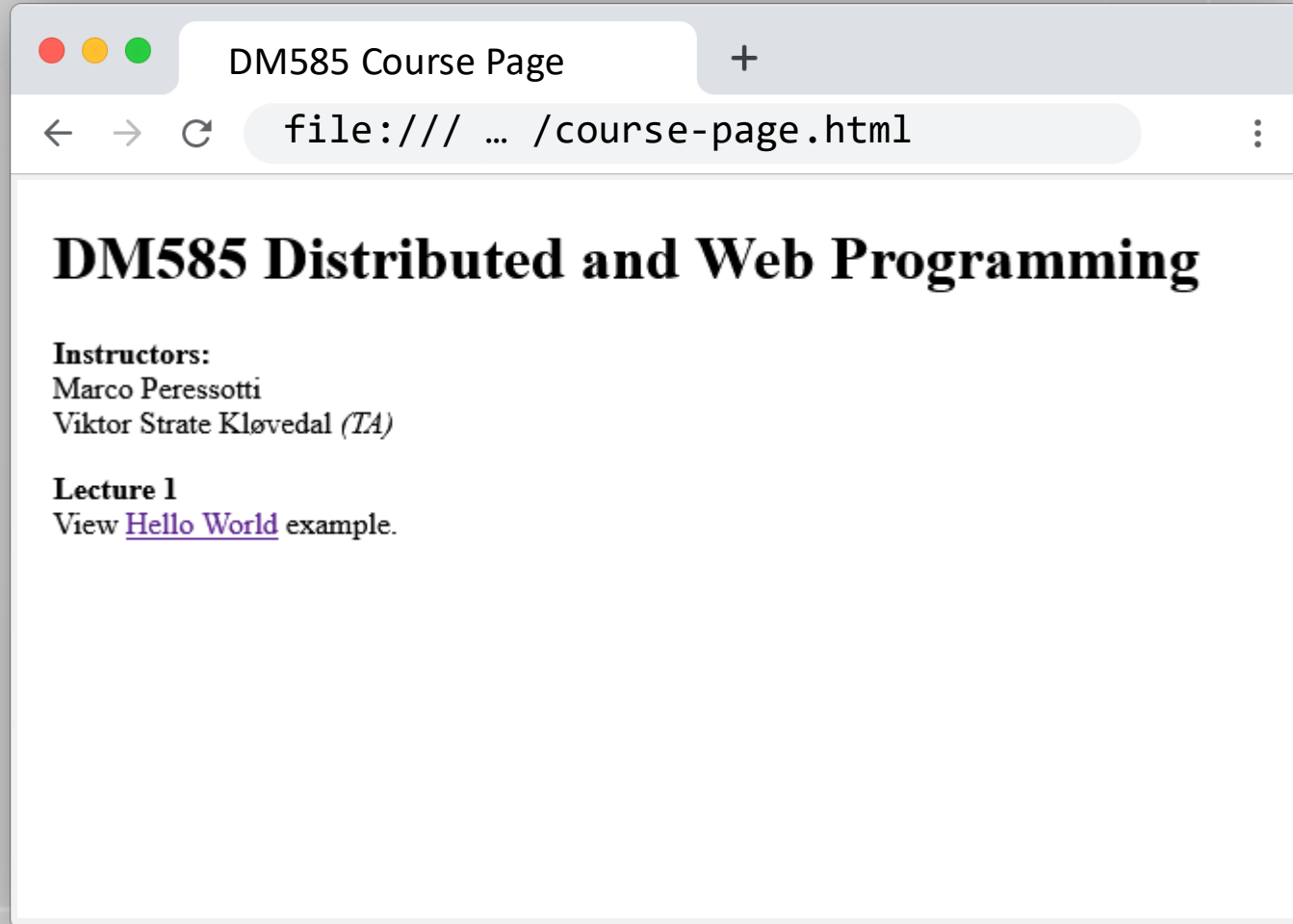
`</p>`

Some HTML elements

Heading h1,h2,...h6	<code><h1>Moby Dick</h1></code>
Paragraph	<code><p>Call me Ishmael.</p></code>
Line break	<code>since feeling is first
who pays any attention</code>
Image	<code></code>
Link	<code>by H. Melville</code>
Strong (bold)	<code>Be BOLD</code>
Emphasis (italic)	<code>He's my brother and all</code>
Lists	<code>An unoreded list ... </code> <code>An ordred ordered ... </code>
List items	<code>HarpoonRope ... </code>

Exercise: Course page

Let's write some HTML to make the following page:



Course page exercise

- We saw that HTML whitespace collapses into one space and needed **
**

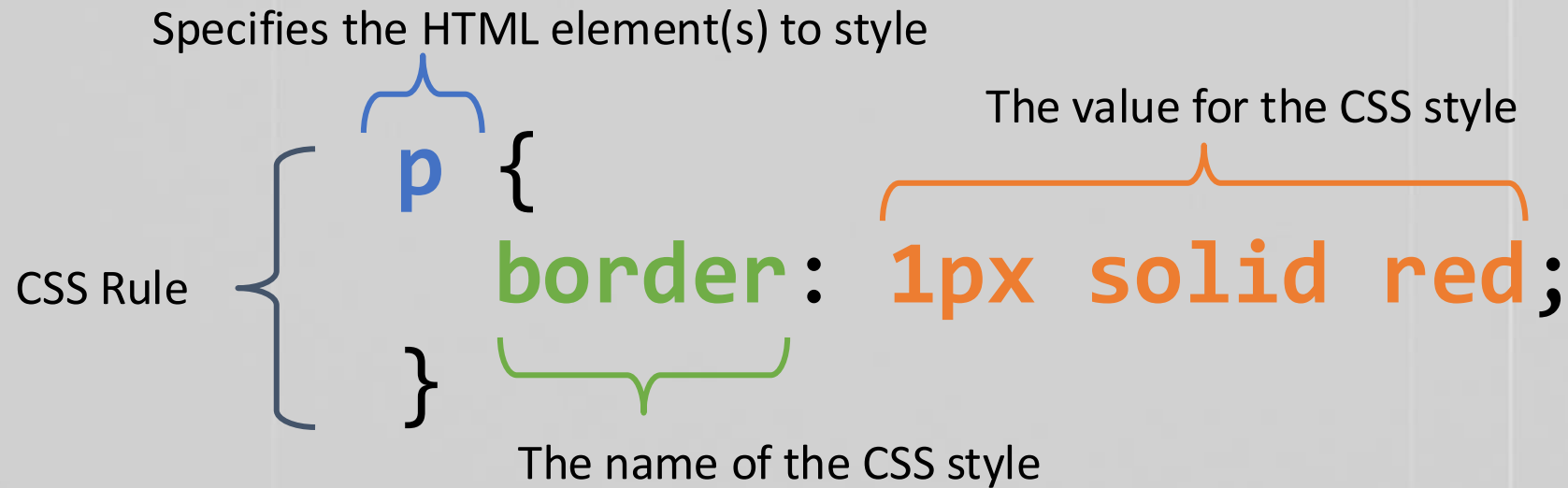
```
<h1>DM585 Distributed and Web Programming</h1>  
<strong>Instructors:</strong><br/>  
Marco Peressotti<br/>  
Viktor Strate Kløvedal <em>(TA)</em>
```

- Except weirdly the **<h1>** heading was on a line of its own, and **** was not.
- 🙄 weird... 😂 but it works... let's think about it later

What is CSS?

CSS: **C**ascading **S**tyle **S**heets

- Describes the **appearance** and **layout** of a web page
- Composed of CSS **rules**, which define sets of styles



Some CSS properties

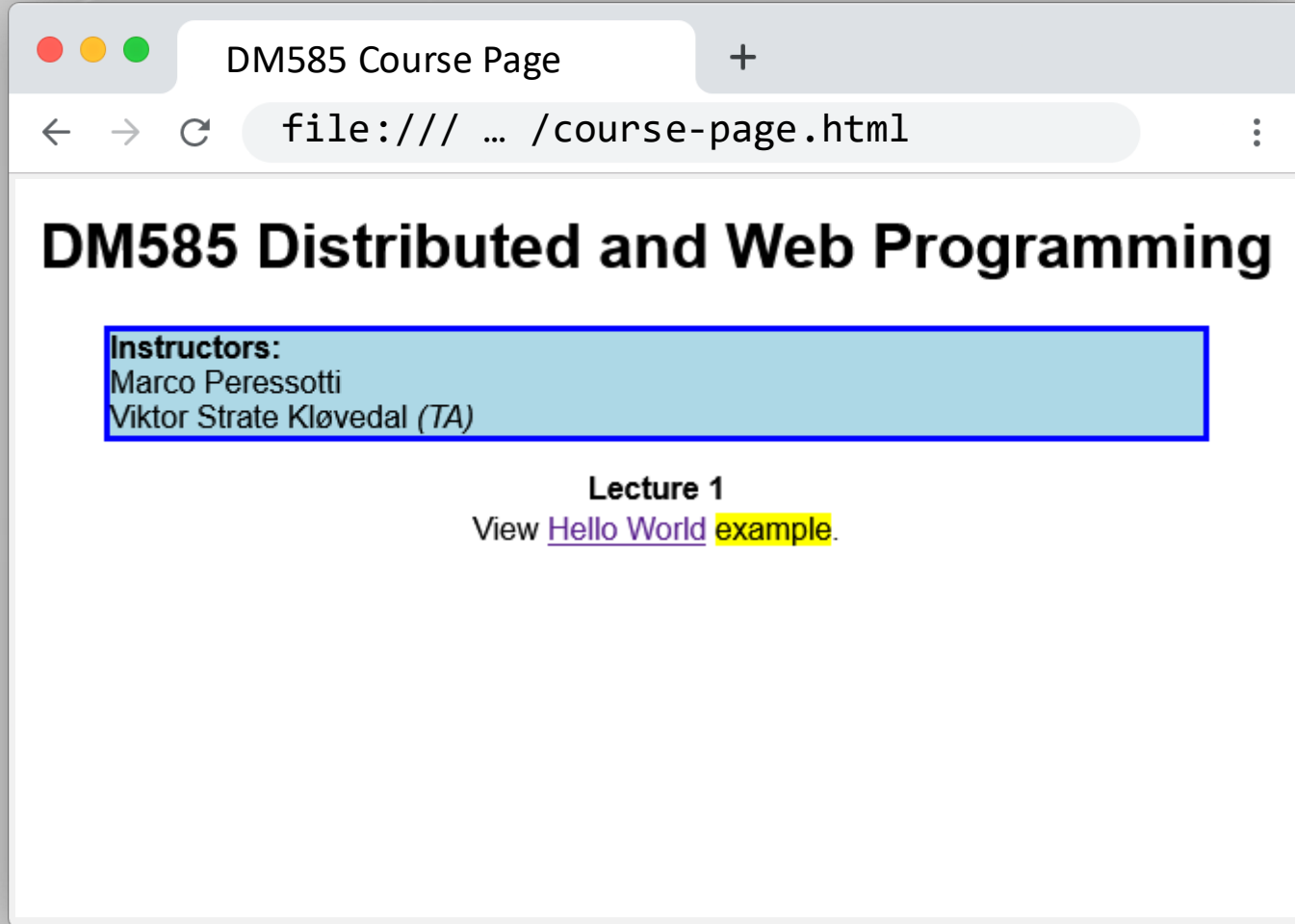
There are over [500 CSS properties](#)! Here are a few:

Font face (mdn)	font-family: Helvetica;
Font color (mdn)	color: gray;
Background color (mdn)	background-color: red;
Border (mdn)	border: 3px solid green;
Text alignment (mdn)	text-align: center;

[Mozilla Developer Network](#) (MDN) is one of the best references for HTML elements and CSS properties.

Exercise: course page

Let's write some CSS to make our page look like this:



- Everything is in Helvetica
- The header is centred
- The “instructors” box has a
 - Border: blue solid 3px
 - Background colour: lightblue
 - Is centred.
 - Content is left-aligned
- The “lectures” block is centred
- The word “example” is highlighted in yellow.

Exercise debrief

- Some key techniques:
 - Add/remove invisible containers in HTML to select groups of elements in CSS.
 - Apply styles to the parent/ancestor element to style the parent and all its children. (Will talk more about this later.)
- But we encountered some issues...
 - `text-align: centre;` didn't work on `<strong>`, `<a>`, ...
 - The box is too wide
 - How do we centre a box?
 - ...

Exercise debrief

- Some key techniques:
 - Add/remove invisible containers in HTML to select groups of elements in CSS.
 - Apply styles to the parent/ancestor element to style the parent and all its children. (Will talk more about this later.)
- But we encountered some issues...
 - `text-align: centre;` didn't work on `<strong>`, `<a>`, ...
 - The box is too wide
 - How do we centre a box?
 - ...

How do we go from this...

DM585 Distributed and Web Programming

Instructors:

Marco Peressotti
Viktor Strate Kløvedal (TA)

Lecture 1

View [Hello World](#) example.



...to this?

DM585 Distributed and Web Programming

Instructors:

Marco Peressotti
Viktor Strate Kløvedal (TA)

Lecture 1

View [Hello World](#) example.



And now...

Block and Inline

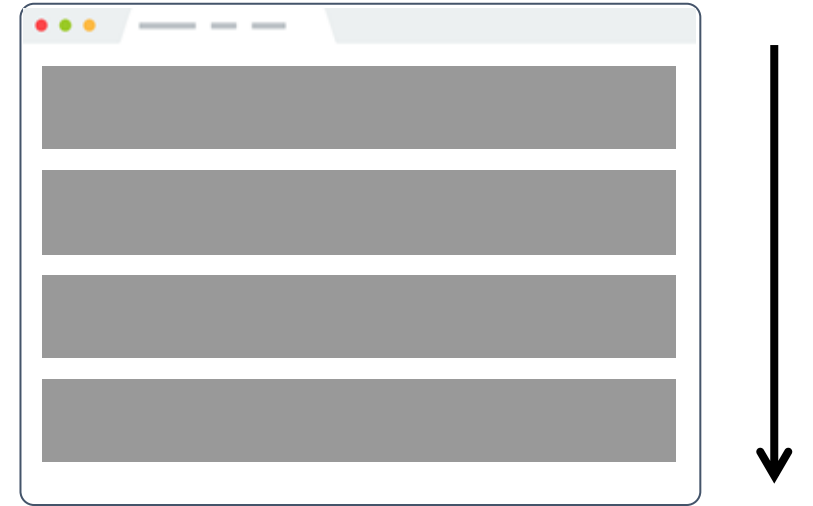
Types of HTML elements

Each HTML element is categorized by the HTML spec into one of three(*ish*) categories:

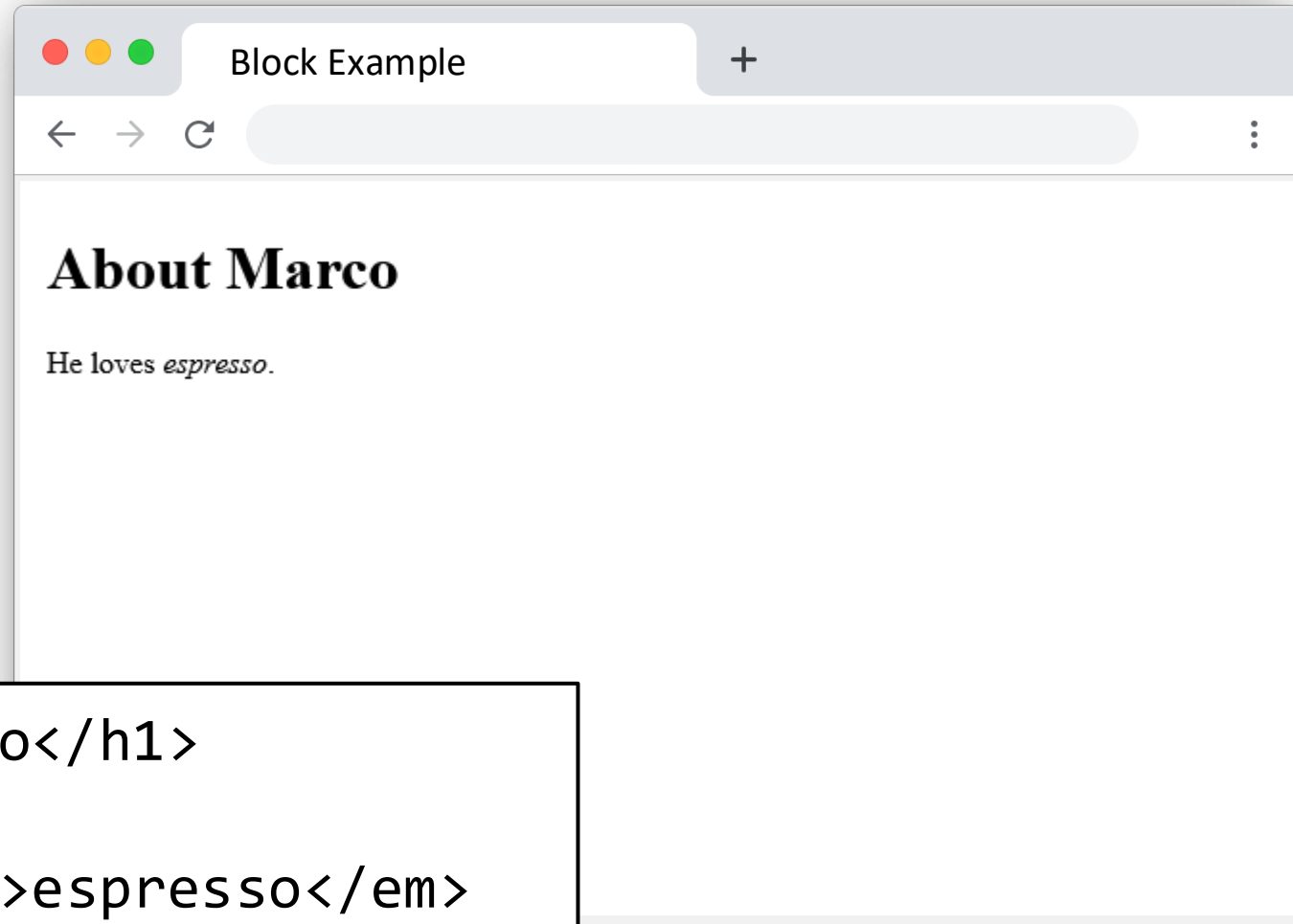
1. **block**: large blocks of content, has height and width
`<p>`, `<h1>`, `<blockquote>`, `<ol>`, `<ul>`, `<table>`
2. **inline**: small amount of content, no height or width
`<a>`, `<em>`, `<strong>`, `<br>`
 - a. **inline block**: inline content with height and width
`<img>`
3. **metadata**: information about the page, usually not visible
`<title>`, `<link>`, `<meta>`

Block elements

- Take up the full width of the page (**flows top to bottom**)
- Have a height and width
- Can have block or inline elements as children
- Examples: `<h1>`, `<h2>`, `<h3>`, `<p>`, `<blockquote>`, `<ol>`, `<ul>`, `<table>`

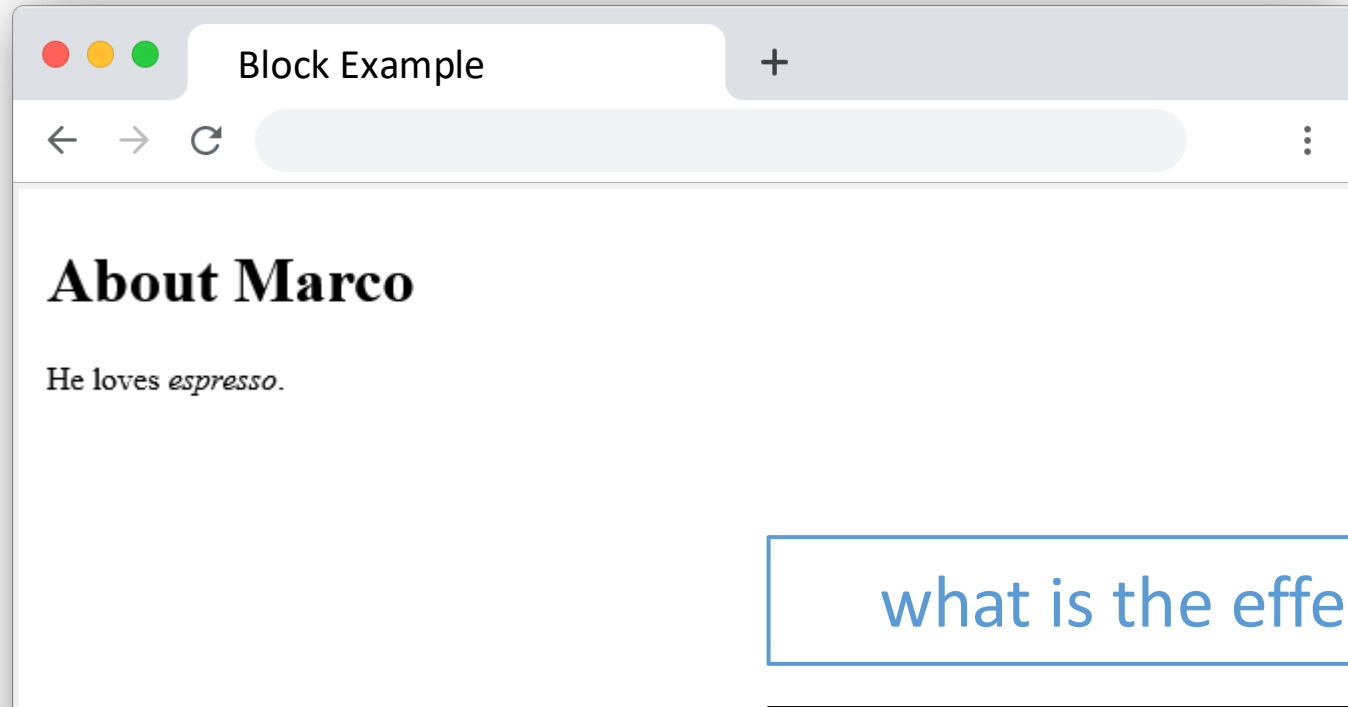


Example: block



```
<h1>About Marco</h1>
<p>
  He loves <em>espresso</em>
</p>
```

Example: block

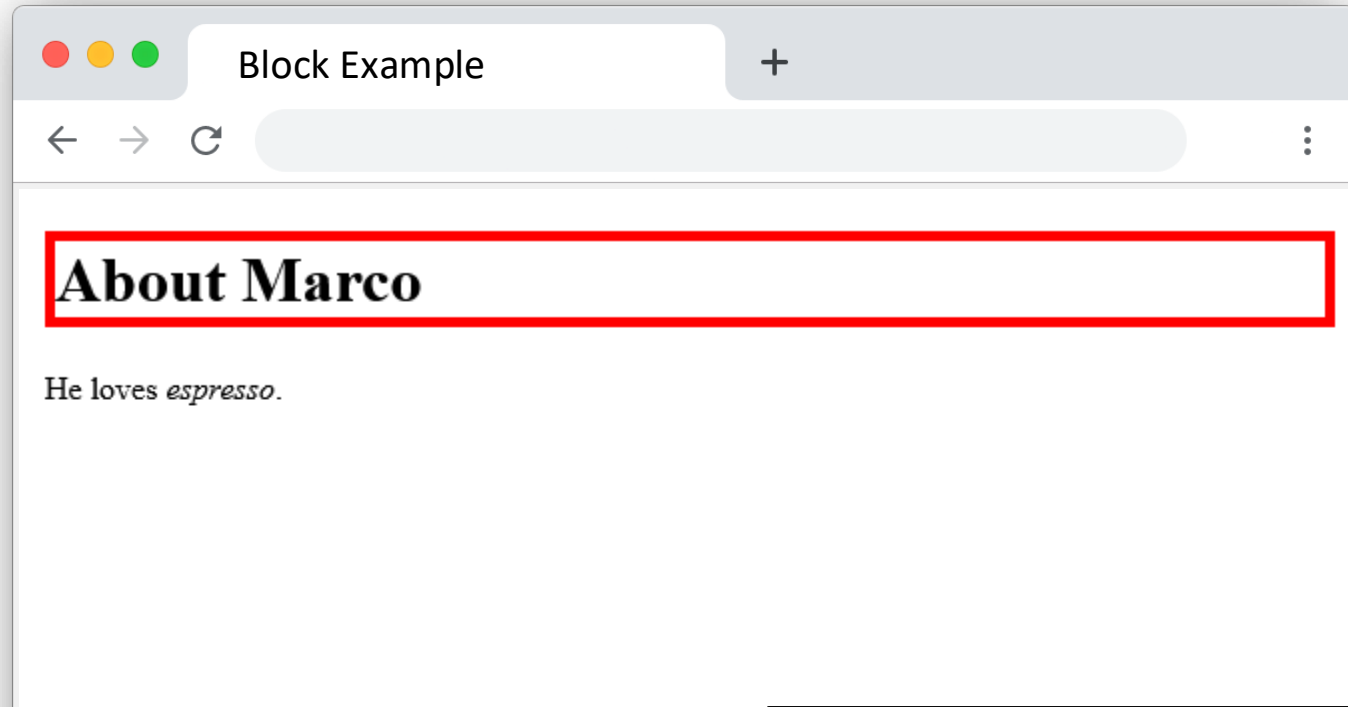


what is the effect of this rule?

```
<h1>About Marco</h1>
<p>
  He loves <em>espresso</em>
</p>
```

```
h1 {
  border: 5px solid red;
}
```

Example: block



```
<h1>About Marco</h1>
<p>
  He loves <em>espresso</em>
</p>
```

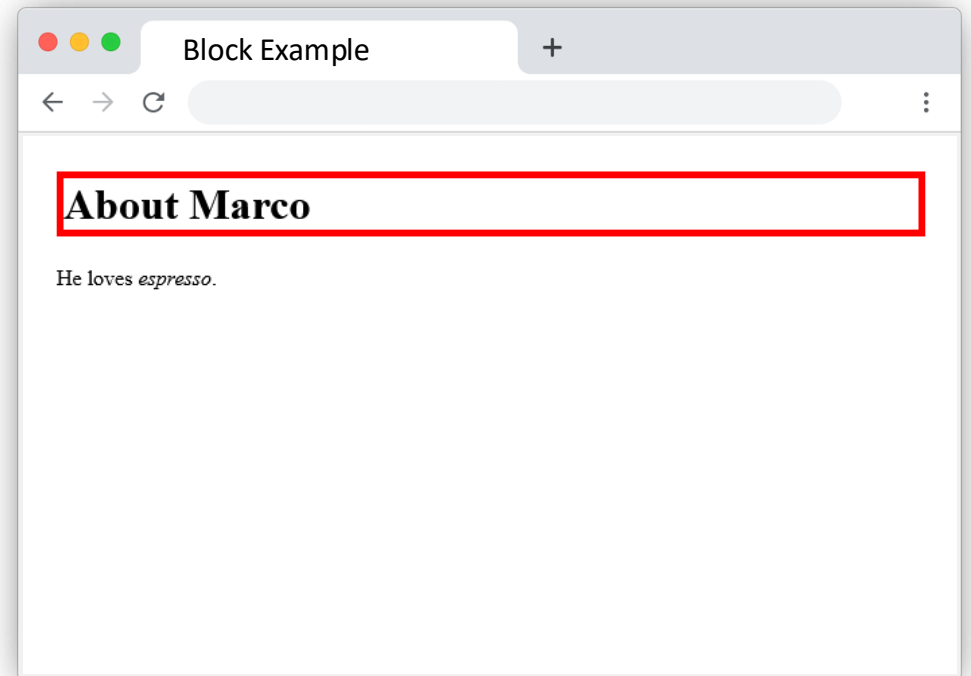
```
h1 {
  border: 5px solid red;
}
```

Block elements have default width

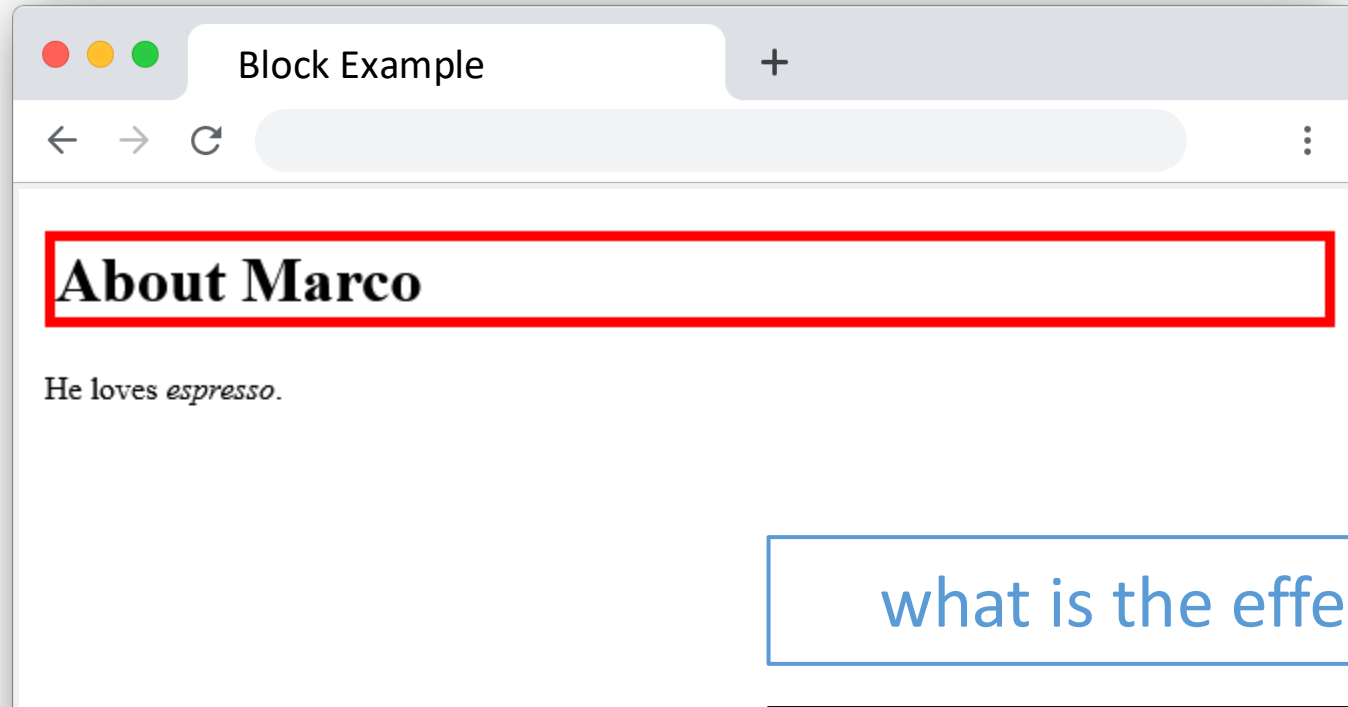
```
h1 {  
  border: 5px solid red;  
}
```

```
<h1>About Marco</h1>  
<p>  
  He loves <em>espresso</em>  
</p>
```

- `<h1>` is block-level, so it extends the **full width** of the page by **default**
- Note how block-level elements (`h1`, `p`) flow top to bottom



Example: block

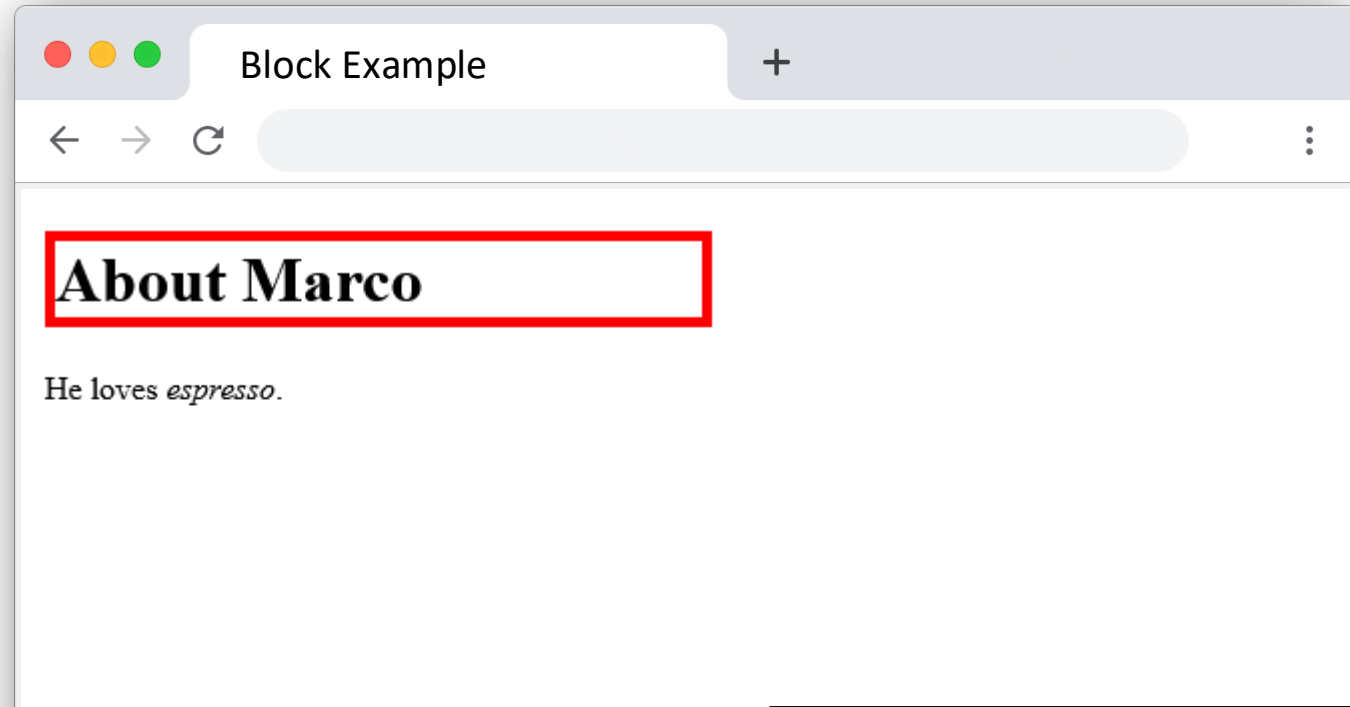


what is the effect of this rule?

```
<h1>About Marco</h1>
<p>
  He loves <em>espresso</em>
</p>
```

```
h1 {
  border: 5px solid red;
  width: 50%;
}
```

Example: block



```
<h1>About Marco</h1>
<p>
  He loves <em>espresso</em>
</p>
```

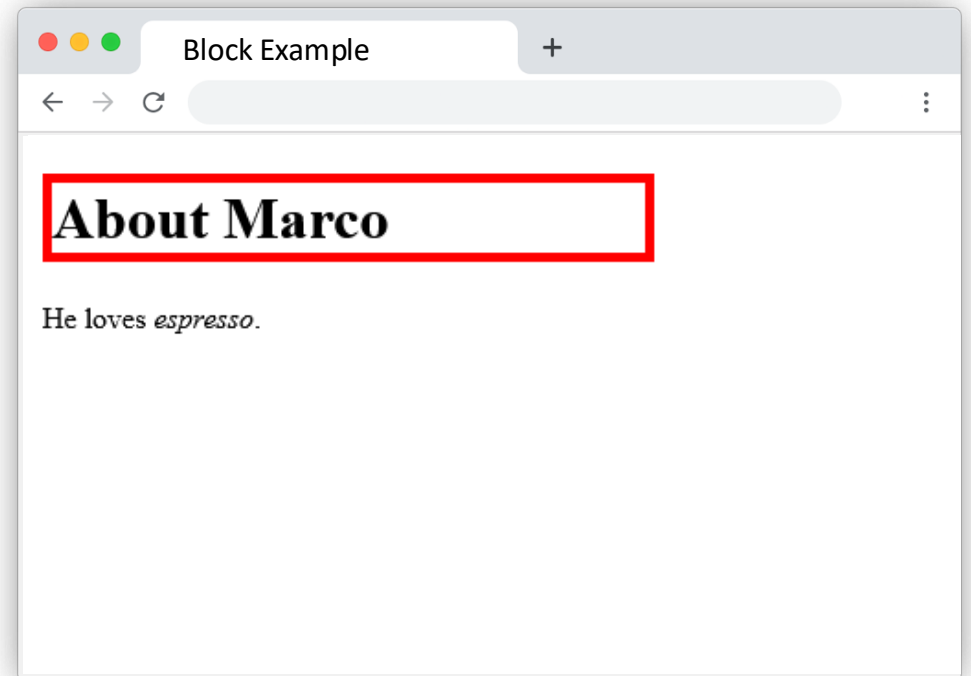
```
h1 {
  border: 5px solid red;
  width: 50%;
}
```

Block elements can have custom width

```
h1 {  
  border: 5px solid red;  
  width: 50%;  
}
```

```
<h1>About Marco</h1>  
<p>  
  He loves <em>espresso</em>  
</p>
```

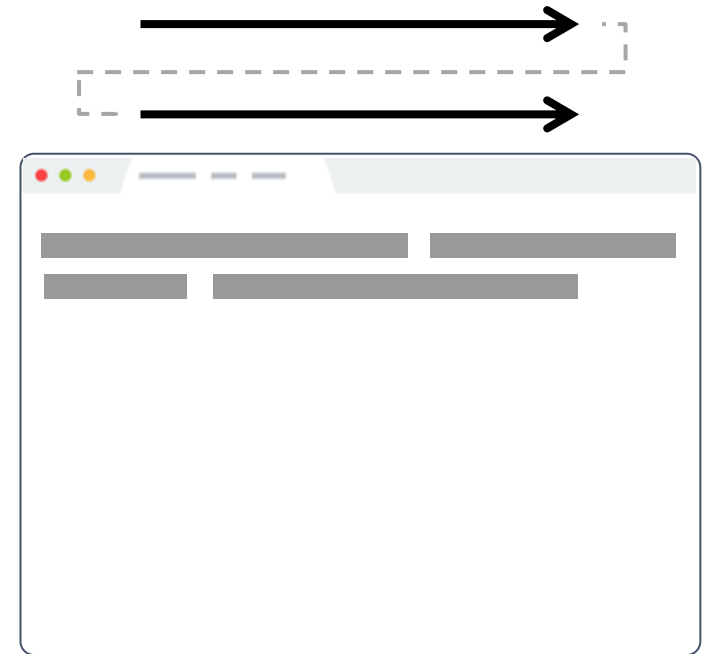
- `<h1>` is block-level, so its **width** **can** be modified
- Block-level elements still flow top to bottom



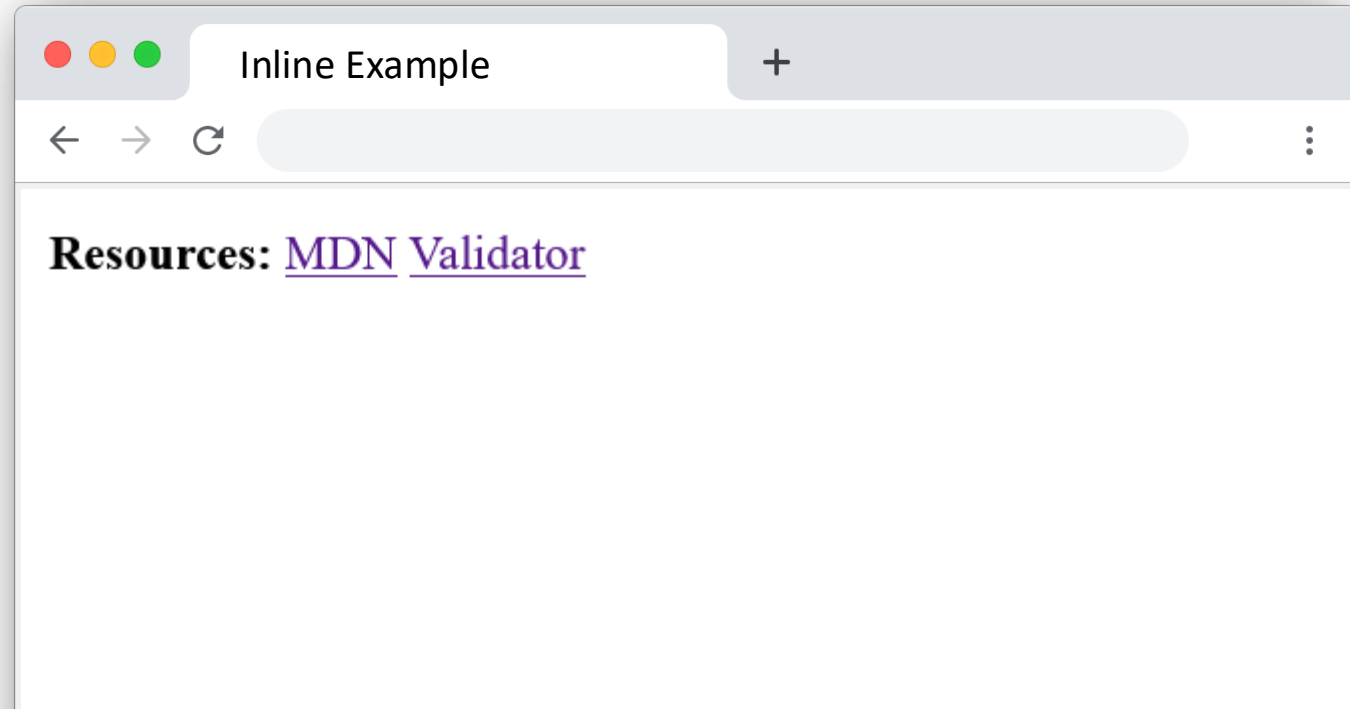
Inline elements

- Take up only as much width as needed
(flow left to right)
- **Cannot** have height and width
- **Cannot** have a block element child
- **Cannot** be positioned
 - (i.e. CSS properties like `float` and `position` do not apply to inline elements)
 - Must position its containing block element instead

Examples: `<a>`, `<em>`, `<strong>`, `<br>`

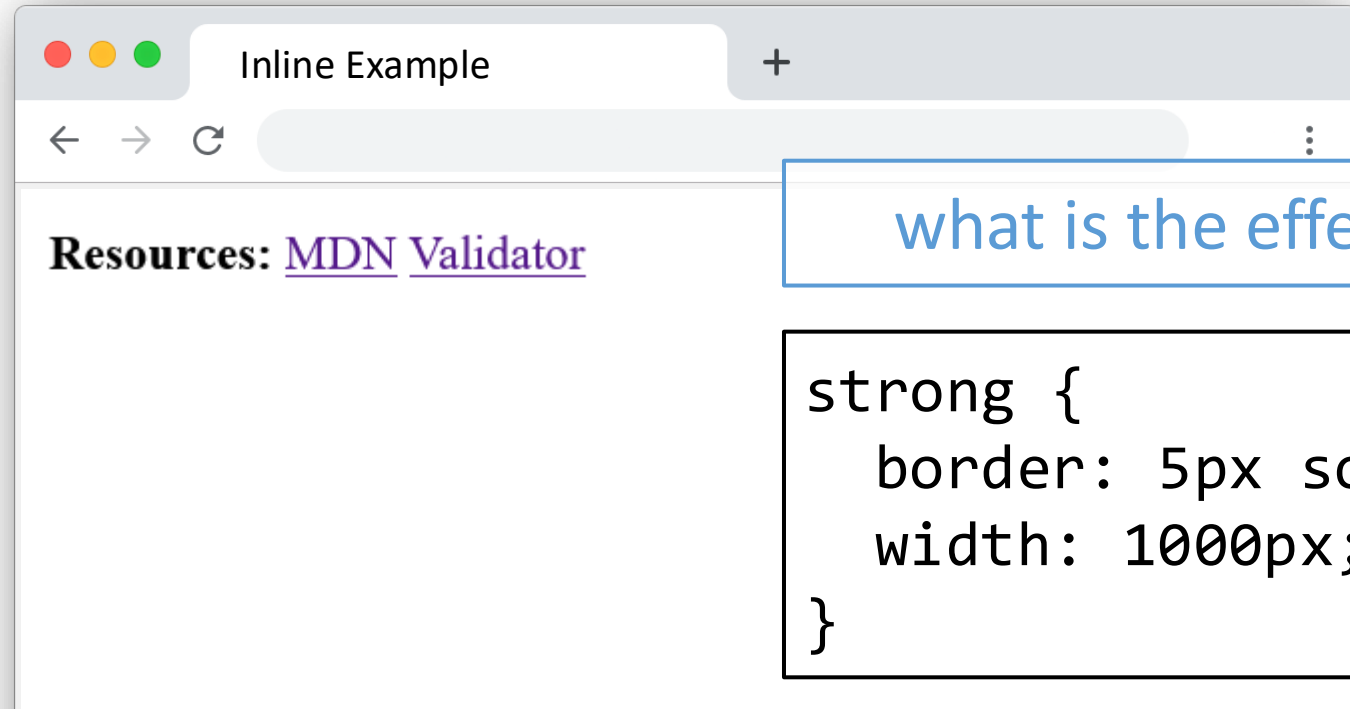


Example: inline



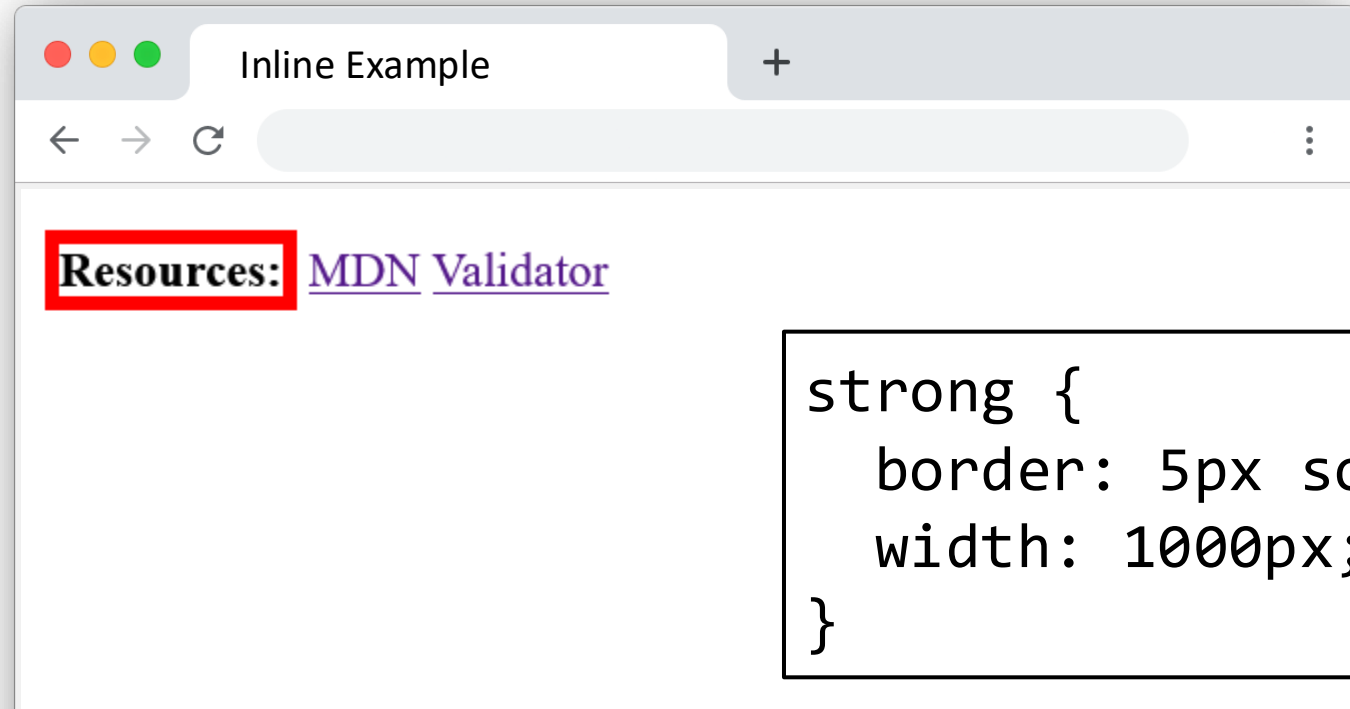
```
<strong>Resources:</strong>  
<a href="https://developer.mozilla.org/">MDN</a>  
<a href="https://validator.w3.org/">Validator</a>
```

Example: inline



```
<strong>Resources:</strong>  
<a href="https://developer.mozilla.org/">MDN</a>  
<a href="https://validator.w3.org/">Validator</a>
```

Example: inline



```
strong {  
  border: 5px solid red;  
  width: 1000px;  
}
```



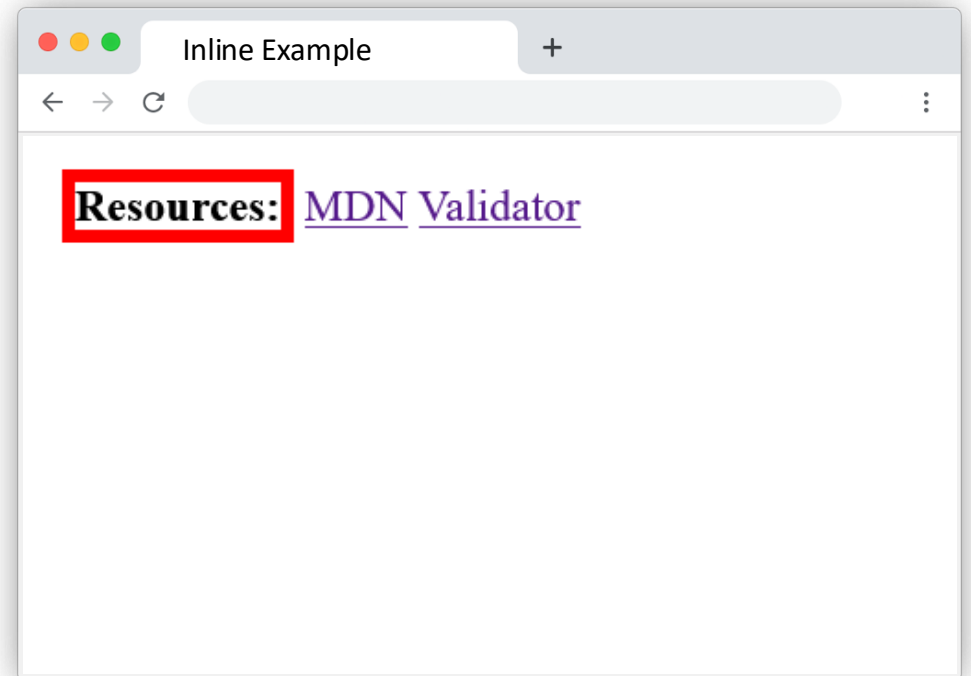
```
<strong>Resources:</strong>  
<a href="https://developer.mozilla.org/">MDN</a>  
<a href="https://validator.w3.org/">Validator</a>
```

Inline elements ignore width

```
strong {  
  border: 5px solid red;  
  width: 1000px; // no effect  
}
```

```
<strong>Resources:</strong>  
<a href="#">MDN</a>  
<a href="#">Validator</a>
```

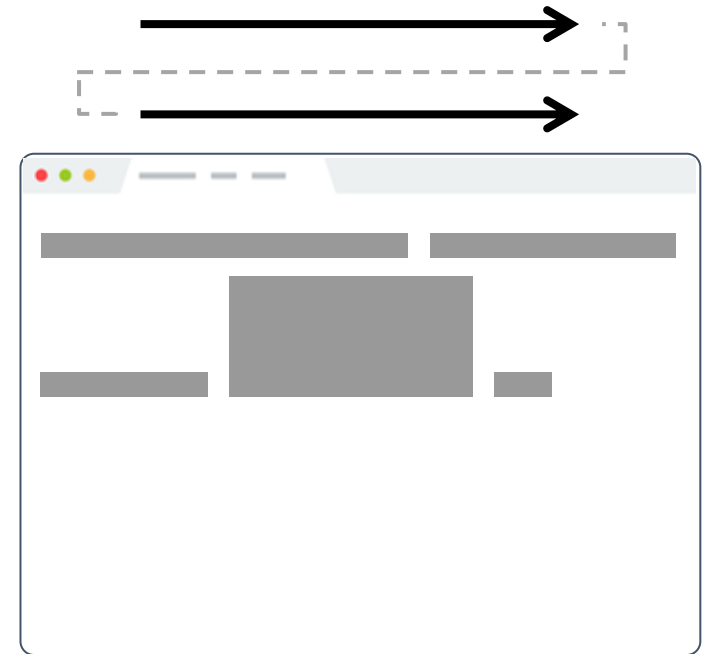
- `<strong>` is inline, its width is given by its content
- inline elements flow left to right



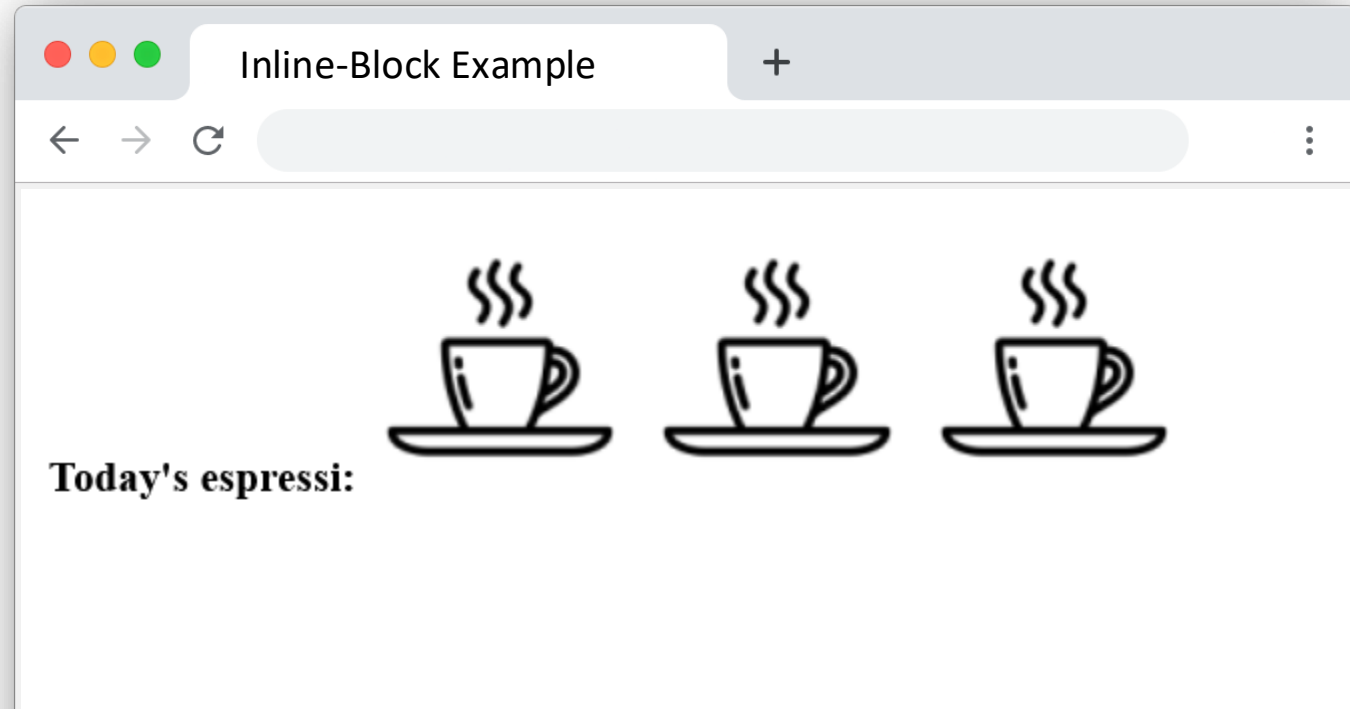
Inline-block

- Width and height are the size of the content, i.e. it takes only as much space as needed (flows left to right)
- **Can** have height and width
- **Can** have a block element as a child
- **Can** be positioned (CSS properties like `float` and `position` can be applied)

Examples: `<img>`



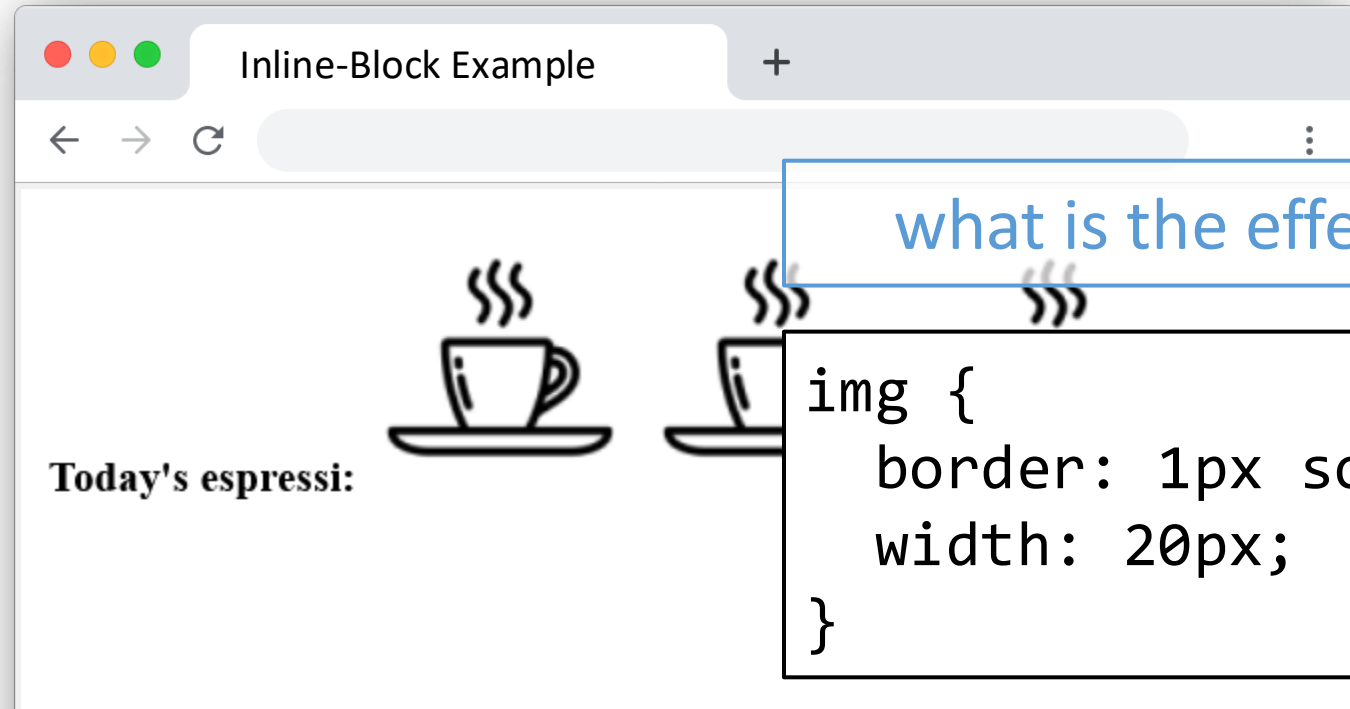
Example: inline-block



```
<strong>Today's espressi:</strong>  
  
  

```

Example: inline-block



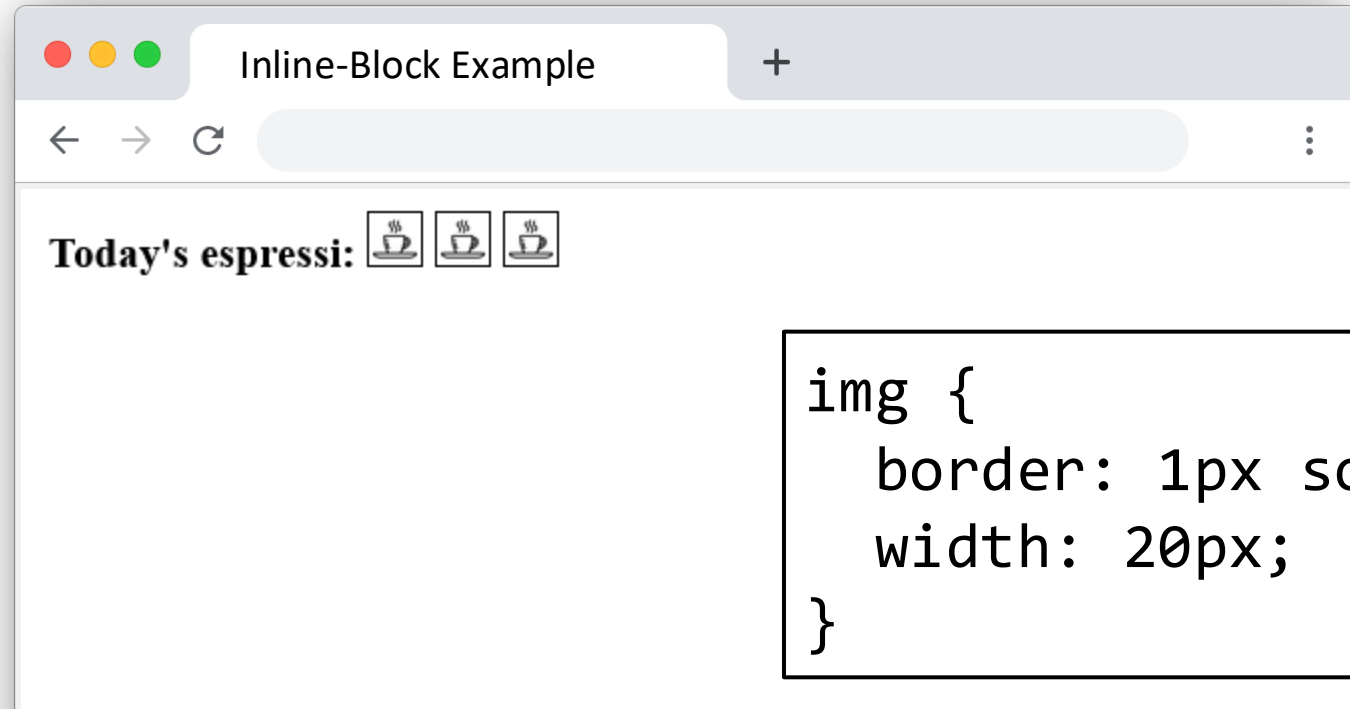
what is the effect of this rule?

```
img {  
  border: 1px solid black;  
  width: 20px;  
}
```

```
<strong>Today's espressi:</strong>  
  
  

```

Example: inline-block



```
img {  
  border: 1px solid black;  
  width: 20px;  
}
```

```
<strong>Today's espressi:</strong>  
  
  

```

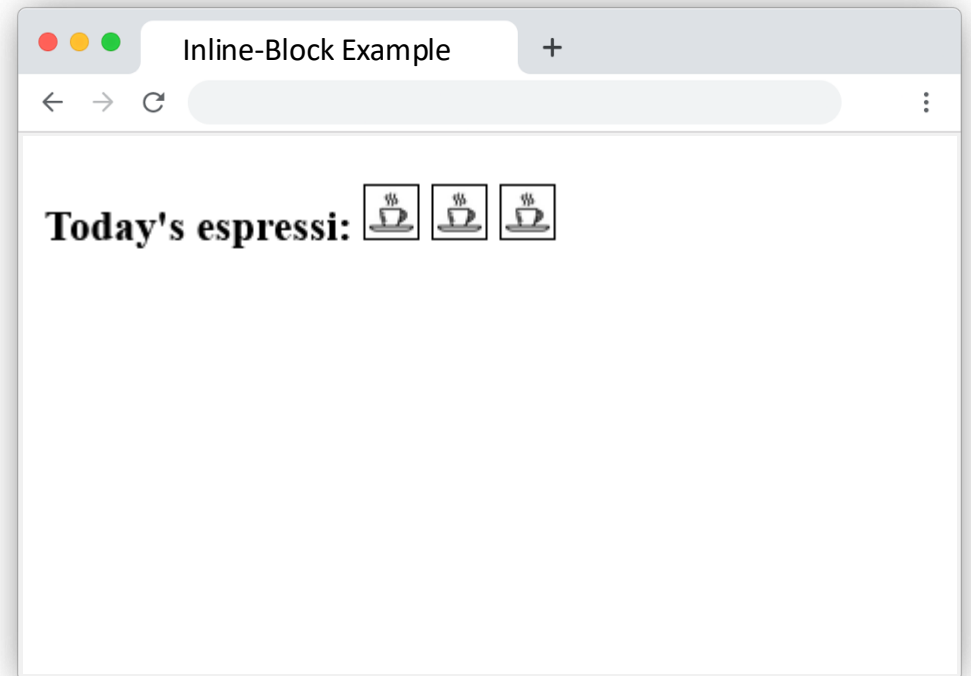
Inline-block elements have size

```
img {  
  border: 1px solid black;  
  width: 20px;  
}
```

```
  
  

```

- `<img>` is inline-block, so its **width** **can** be modified
- inline-block elements flow left to right, like inline elements, so the images follow the text line



The display CSS property

You can change an element's default rendering type by changing the **display** property.

```
p {  
  display: inline;  
}
```

```
a {  
  display: block;  
}
```

Possible values for display:

- block
- inline
- inline-block
- some others: [link](#)

<div> and

Two generic tags with no intended purpose or style:

- <div>: a generic **block** element
- : a generic **inline** element

Prioritise specific tags.

Review

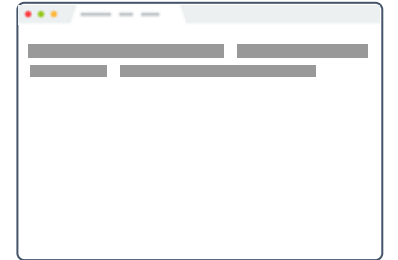
Block

- flows top-to-bottom
- has height and width
- `<p>`, `<h1>`, `<blockquote>`, `<ol>`, `<ul>`, `<table>`, `<div>`



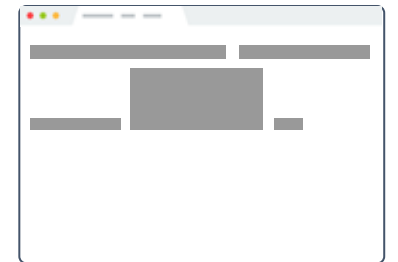
Inline

- flows left-to-right
- does not have height and width
- `<a>`, `<em>`, `<strong>`, `<br>`, `<span>`



Inline block

- flows left-to-right
- has height and width equal to size of the content
- `<img>`



Can we finish the course page exercise?

Exercise debrief

- Some key techniques:
 - Add/remove invisible containers in HTML to select groups of elements in CSS.
 - Apply styles to the parent/ancestor element to style the parent and all its children. (Will talk more about this later.)
- But we encountered some issues...
 - `text-align: centre;` didn't work on `<strong>`, `<a>`, ...
 - The box is too wide
 - How do we centre a box?
 - ...

How do we go from this...

DM585 Distributed and Web Programming

Instructors:
Marco Peressotti
Viktor Strate Kløvedal (TA)

Lecture 1

View [Hello World](#) example.



...to this?

DM585 Distributed and Web Programming

Instructors:
Marco Peressotti
Viktor Strate Kløvedal (TA)

Lecture 1

View [Hello World](#) example.

Can we finish the course page exercise?

Exercise debrief

- Some key techniques:
 - Add/remove invisible containers in HTML to select groups of elements in CSS.
 - Apply styles to the parent/ancestor element to style the parent and all its children. (Will talk more about this later.)
- But we encountered some issues...
 - `text-align: centre;` didn't work on `<strong>`, `<a>`, ...
 - The box is too wide
 - How do we centre a box?
 - ...

How do we go from this...

DM585 Distributed and Web Programming

Instructors:
Marco Peressotti
Viktor Strate Kløvedal (TA)

Lecture 1

View [Hello World](#) example.



...to this?

DM585 Distributed and Web Programming

Instructors:
Marco Peressotti
Viktor Strate Kløvedal (TA)

Lecture 1

View [Hello World](#) example.

Well...almost

CSS Selectors and Inheritance

CSS selectors

Name	Syntax	Description
Element selector (the one we have used so far)	p	Select all <p> elements

CSS selectors

There are 3 basic types CSS selectors

Name	Syntax	Description
Element selector (the one we have used so far)	p	Select all <p> elements
ID selector	#abc	Select the element with attribute id="abc"
Class selector	.abc	Select all elements with attribute class="abc"

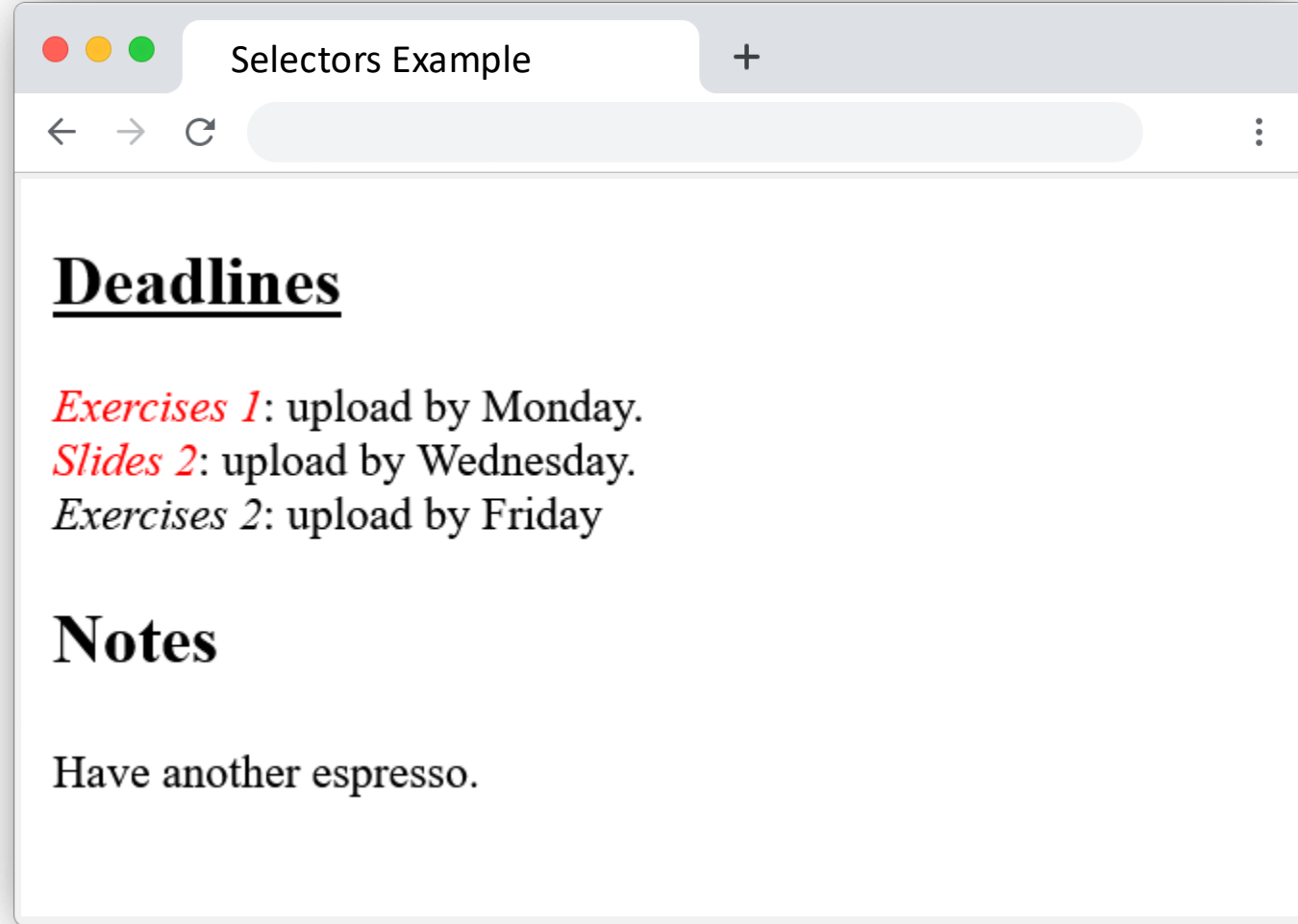
```
<h2 id="deadlines">Deadlines</h2>
<em class="past">Exercises 1</em>: upload by Monday.<br/>
<em class="past">Slides 2</em>: upload by Wednesday.<br/>
<em>Exercises 2: upload by Friday</em>
```

CSS selectors

```
<h2 id="deadlines">Deadlines</h2>
<em class="past">Exercises 1</em> ...
<em class="past">Slides 2</em> ...
<em>Exercises 2</em> ...
```

```
<h2>Notes</h2>
...
```

```
#deadlines {
  text-decoration: underline;
}
.past {
  color: red;
}
```



Class and ID

- **class** and **id** are special HTML attributes that can be used on any HTML element
 - **class**: values can be used on **one or more** elements; identifies a **collection** of elements
 - **id**: values can be used on **exactly one** element per page; identifies **one unique** element
- Can apply multiple classes by space-separating them:
`<span class="todo new urgent">Slides 2</span>`
- Often used with span and div to create generic elements: e.g. `<span class="alert">` is like creating an "alert/highlight" element
- Do not overuse span, div, class, ...
 - `<p>` has an unambiguous semantics
 - `<div class="paragraph">` does not

CSS selectors and combinators

Name	Syntax	Description
Element selector	p	Select all <p> elements
ID selector	#abc	Select the element with id="abc"
Class selector	.abc	Select all elements with class="abc"
Element and class/id	p.abc	Select all <p> elements with class="abc"
	p#abc	Select all <p> elements with id="abc"
Descendent combinator	p em	Select all descendants of a <p>
Union combinator	h1, h2	Select all <h1> and all <h2>

Exercise

Explain the following selectors

1. `li.abc`
2. `li .abc`
3. `ul li`
4. `ul, li`
5. `div#abc p, h2`
6. `div.abc, .abc`

p	Select all <p> elements
#abc	Select the element with id="abc"
.abc	Select all elements with class="abc"
p.abc	Select all <p> elements with class="abc"
p#abc	Select all <p> elements with id="abc"
p em	Select all descendants of a <p>
h1, h2	Select all <h1> and all <h2>

More CSS combinators

Element selector	p	Select all <p> elements
ID selector	#abc	Select the element with id="abc"
Class selector	.abc	Select all elements with class="abc"
Element and class/id	p.abc	Select all <p> elements with class="abc"
	p#abc	Select all <p> elements with id="abc"
Descendent combinator	p em	Select all descendants of a <p>
Union combinator	h1, h2	Select all <h1> and all <h2>
Child combinator	div > p	Select all <p> direct children of <div>
Subsequent sibling	h2 ~ p	Select all <p> that follow an <h2>
Next sibling	h2 + p	Select all <p> that directly follow an <h2>

Inheritance

We saw in our examples that CSS styles are inherited from parent to child.

Instead of selecting all elements individually:

```
a, h1, p, strong {  
    font-family: Helvetica;  
}
```

You can style the parent and the children will inherit the styles.

```
body {  
    font-family: Helvetica;  
}
```

You can override this style via specificity:

```
h1, h2 {  
    font-family: Consolas;  
}
```

Inheritance and override

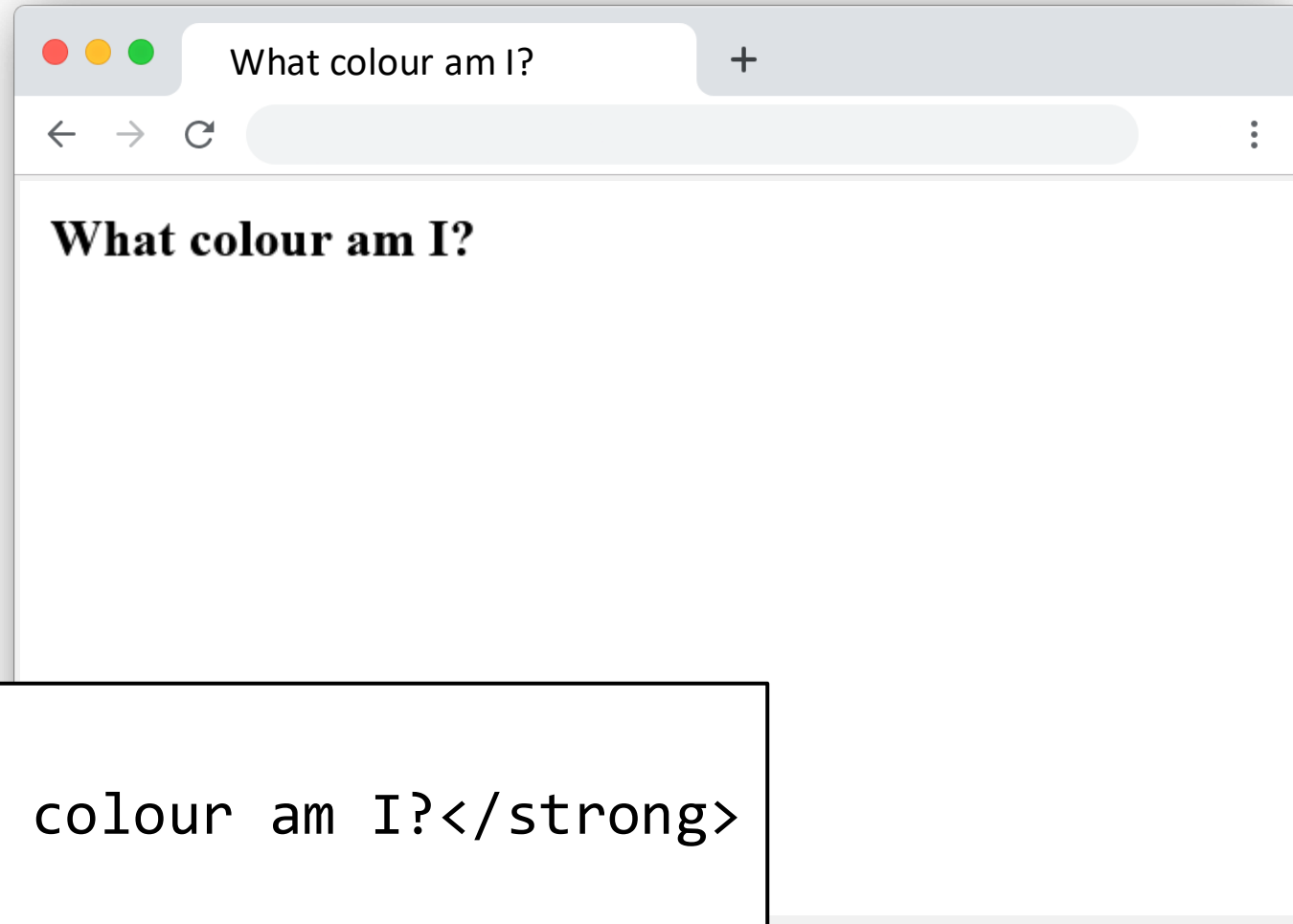
- While many CSS styles are inherited from parent to child, **not all CSS properties are inherited**.
- For instance, `<em>` inherits the `font-family` property, but not `display` (it overrides it).
- There's no rule for what properties are inherited or overridden; the inheritance behaviour defined in the CSS spec.
- You can look it up via MDN, see e.g., [font-family](#) and [display](#).
- Generally, text-related properties are inherited, and layout-related properties are not (but this is not a rule).

```
a {  
  display: block;  
  font-family: Arial;  
}
```

```
<a href="/home">  
  Back to <em>Home</em>  
</a>
```

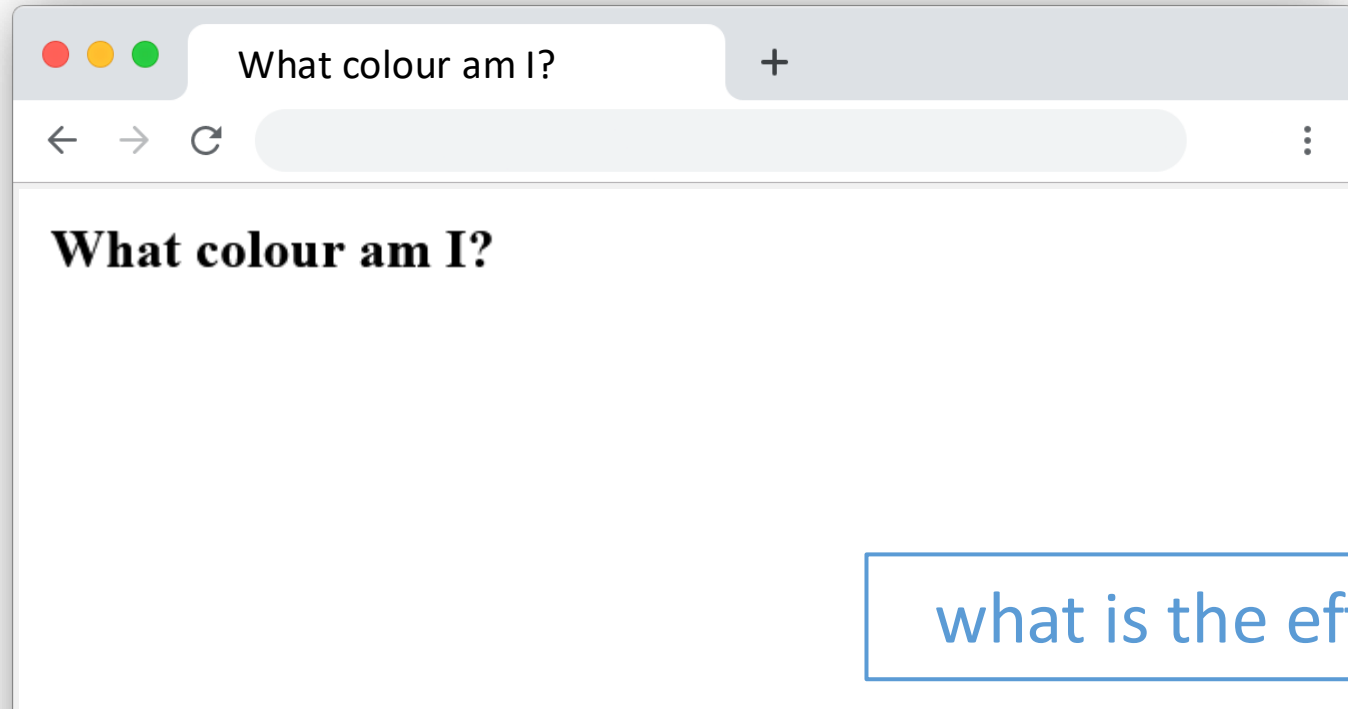
[Back to Home](#)

Exercise: what colour am I?



```
<div>  
  <strong>What colour am I?</strong>  
</div>
```

Exercise: what colour am I?

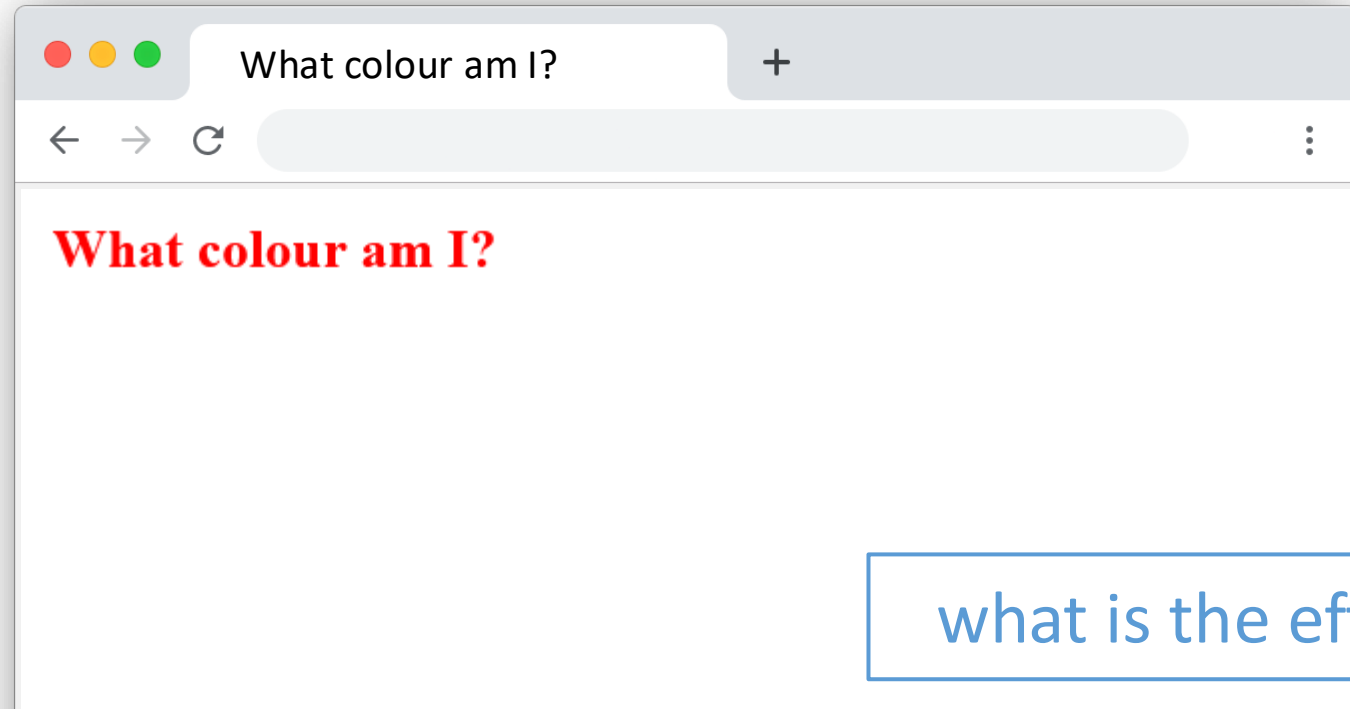


what is the effect of this rule?

```
<div>
  <strong>What colour am I?</strong>
</div>
```

```
div strong { color: red; }
strong { color: blue; }
```

Exercise: what colour am I?



what is the effect of this rule?

```
<div>
  <strong>What colour am I?</strong>
</div>
```

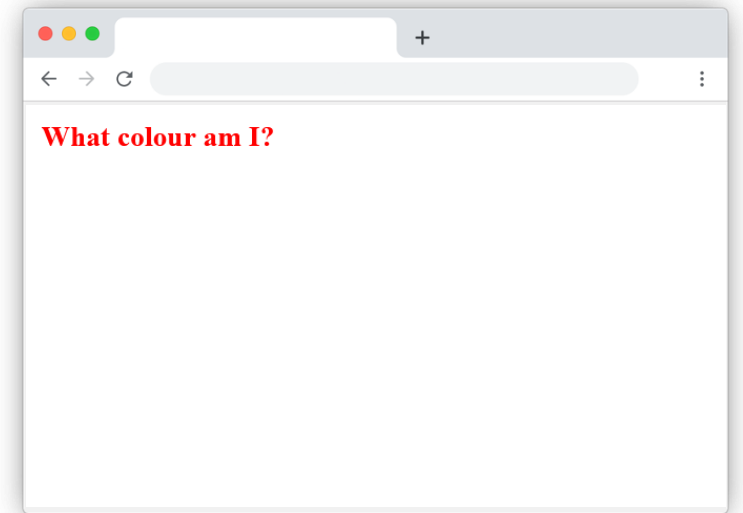
```
div strong { color: red; }
strong { color: blue; }
```

Colliding styles

- A collision happens when more styles can be applied.
- The most specific rule wins([specificity rules](#)):
 - ids are more specific than classes
 - classes are more specific than element names
 - Style rules that directly target elements are more specific than style rules that are inherited

(The process of figuring out what rules apply to an element is called the [cascade](#). This is the “C” in *Cascading* Style Sheets.)

```
div strong { color: red; }  
strong { color: blue; }
```



Pseudo-classes

- pseudo-classes are special keywords you can append to selectors, specifying a state or property of the selector
- Links and pseudo-classes

Syntax	Explanation
a	All anchor tags (links) in all states
a:visited	All anchors linking a visited page
a:link	All anchors linking an unvisited page
a:hover	The style when you hover over an anchor
a:active	The style when you have "activated" an anchor (downclick)

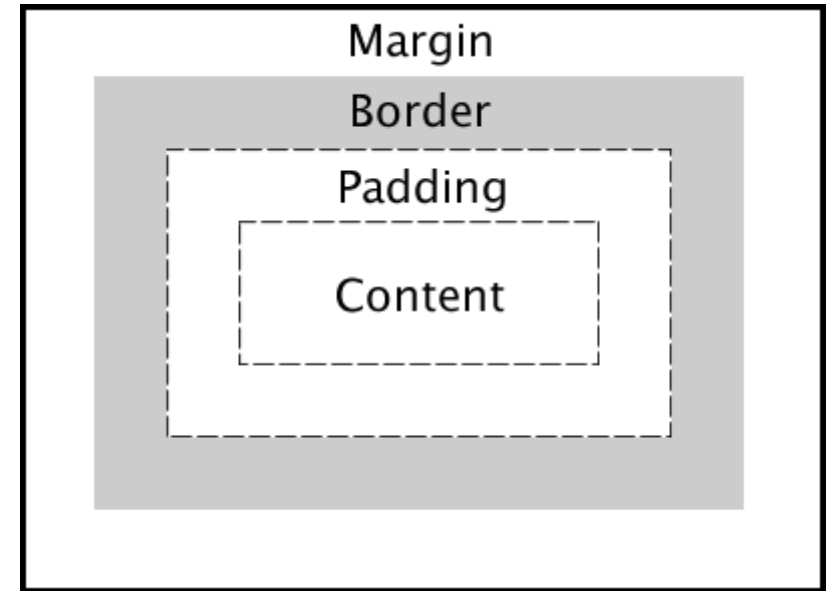
- Some of these (e.g., hover) apply to other elements besides <a>
- There are more [pseudo-classes](#)

CSS Box Model

The CSS Box Model

Every element is composed of 4 layers:

- the element's content
- the **border** around the element's content
- **padding** space between the content and border (inside)
- a **margin** clears the area around border (outside)



Border

We've used the shorthand:

[border](#): *width style color*;

Can also specify each border individually:

`border-top`: *width style color*;

`border-bottom`: *width style color*;

`border-left`: *width style color*;

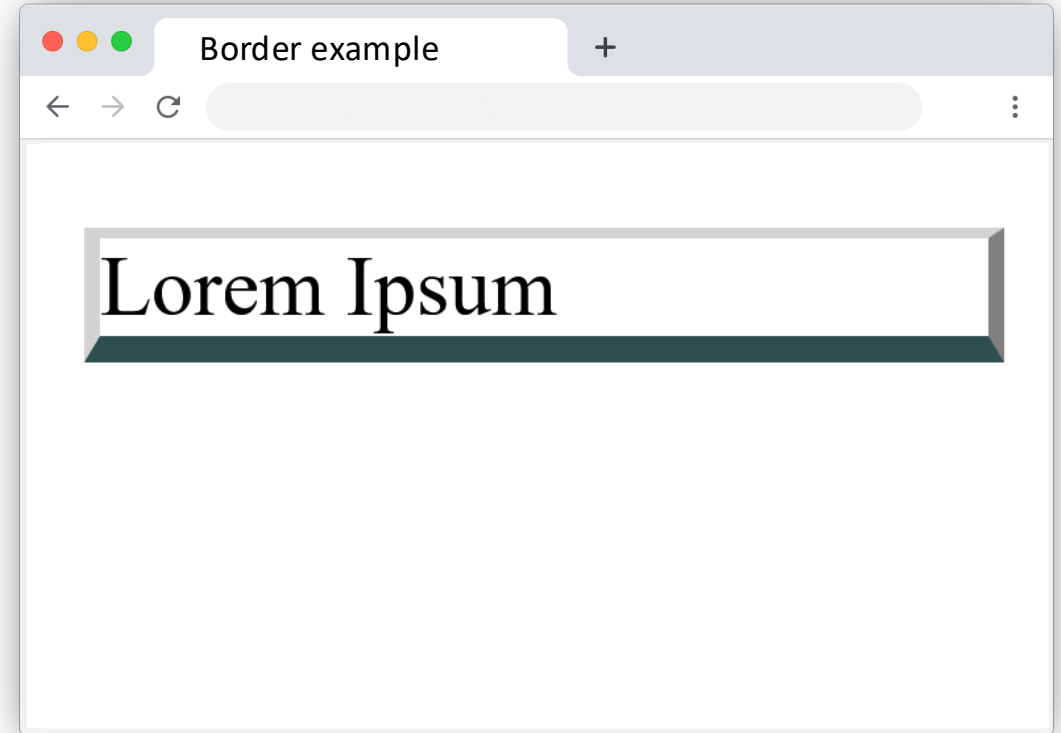
`border-right`: *width style color*;

And can set each property individually:

[border-style](#): *style*;

[border-width](#): *width*;

[border-color](#): *color*;



```
p {  
  border-top: 2px solid lightgray;  
  border-left: 3px solid lightgray;  
  border-right: 3px solid grey;  
  border-bottom: 5px solid darkslategrey;  
}
```

Padding

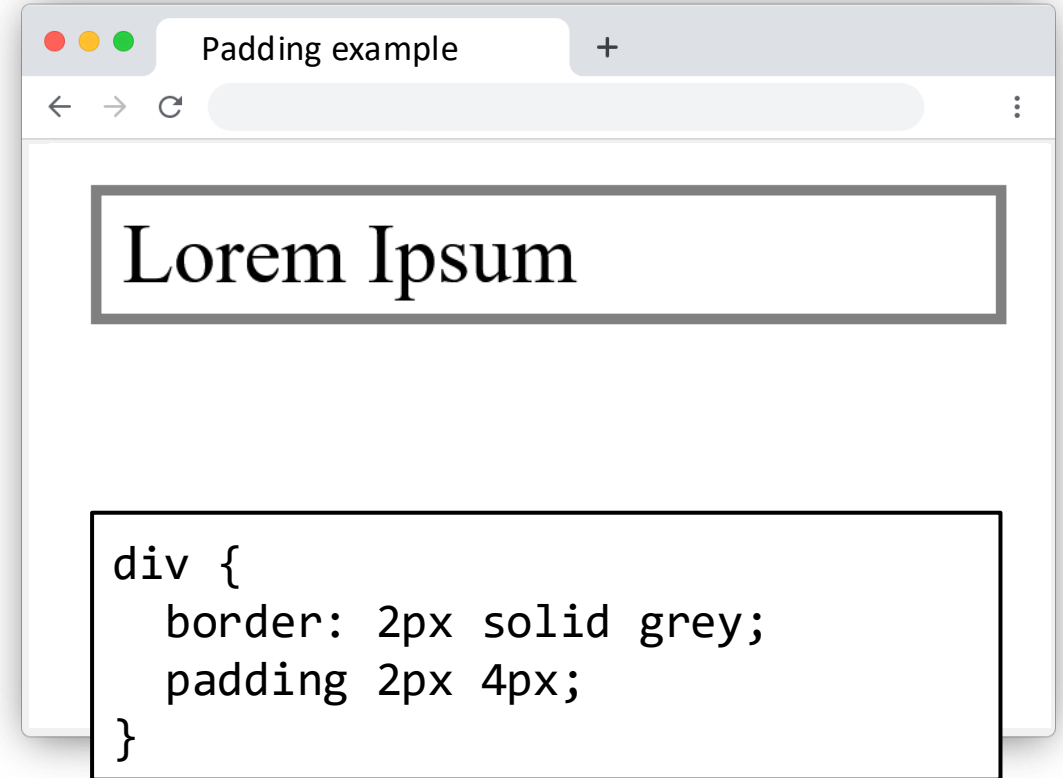
Padding is the space between the border and the content.

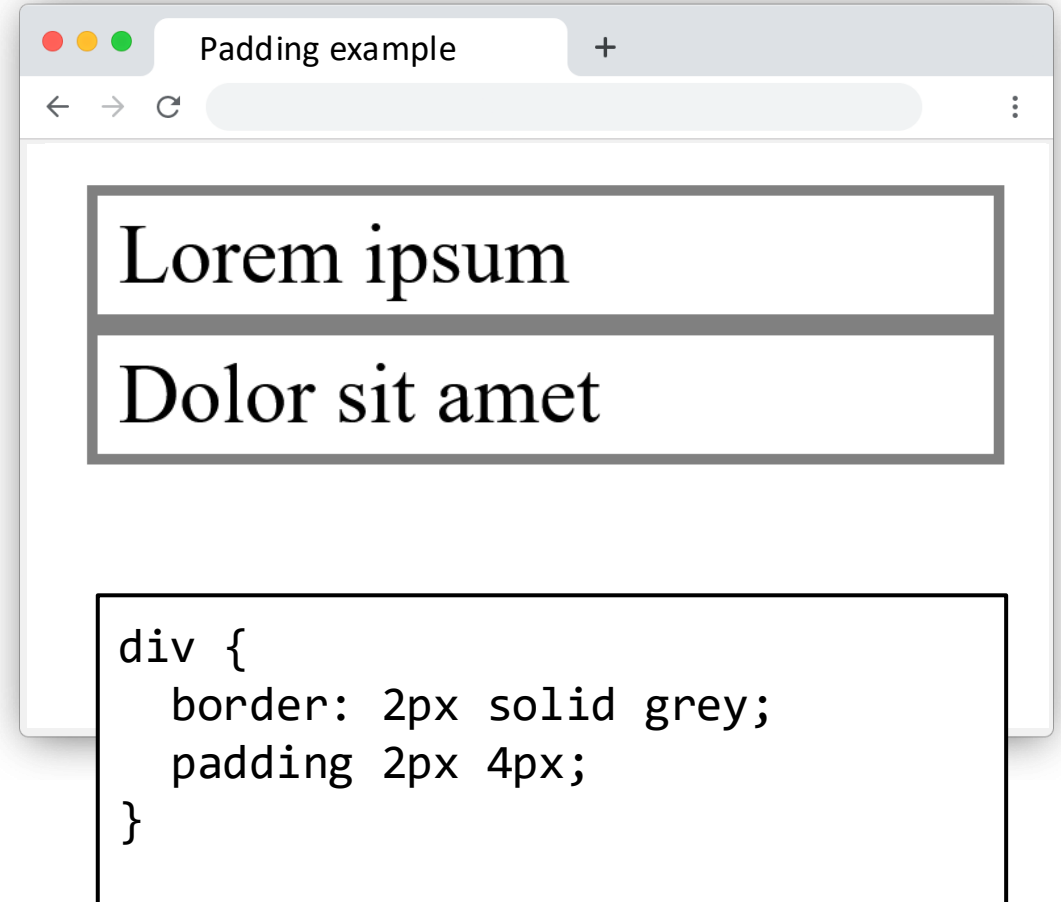
Can specify each direction separately

```
padding-top:    2px;  
padding-bottom: 2px;  
padding-left:   4px;  
padding-right   4px;
```

Or via the shorthand:

```
padding: 2px 4px 2px 4px; // top right bottom left  
padding: 2px 4px;        // top&bottom left&right  
padding: 5px;            // top&bottom&left&right
```





Margin

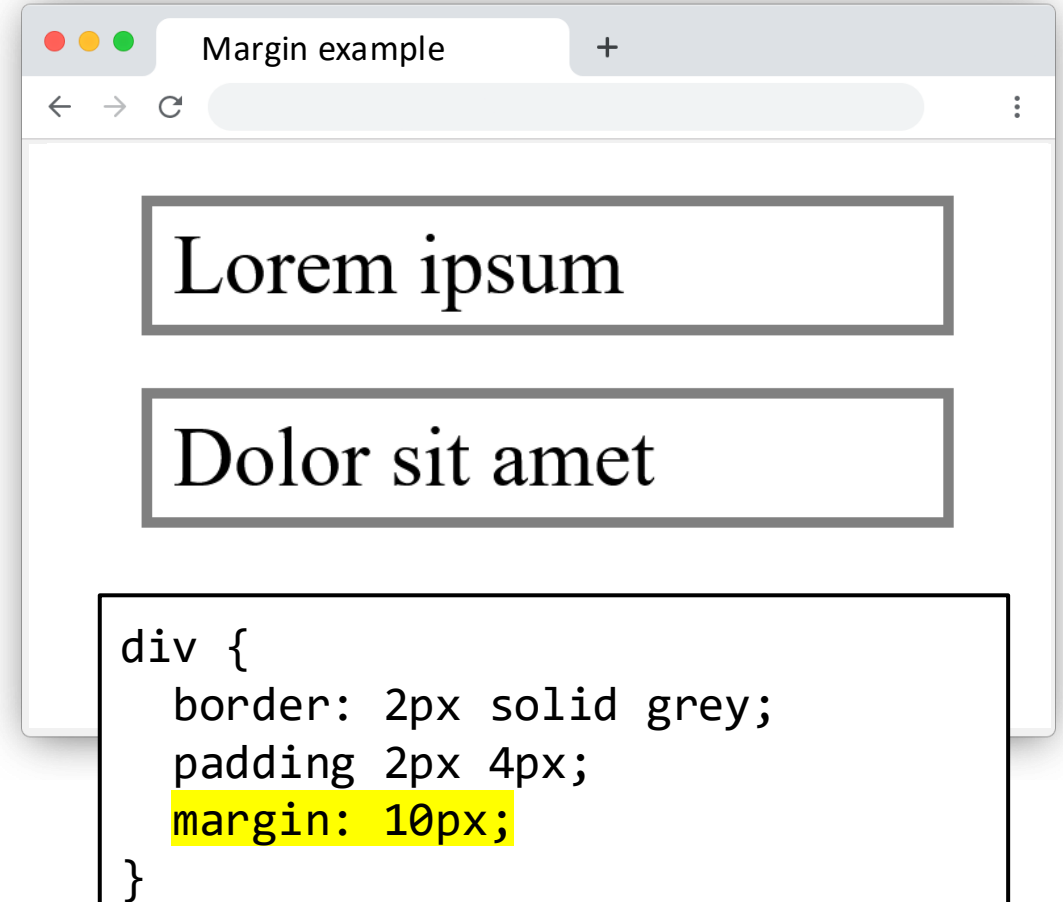
Margin is the space between the border and other elements.

Can specify each direction separately

```
margin-top:    2px;  
margin-bottom: 2px;  
margin-left:   4px;  
margin-right   4px;
```

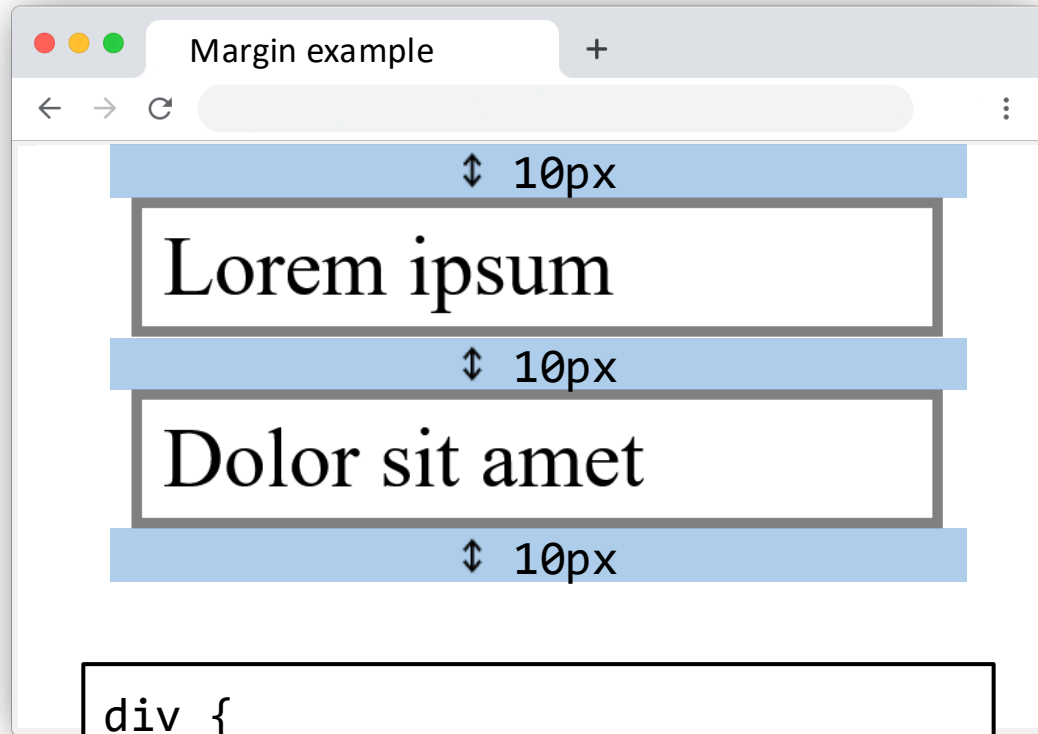
Or via the shorthand:

```
margin: 2px 4px 2px 4px; // top right bottom left  
margin: 2px 4px;         // top&bottom left&right  
margin: 5px;             // top&bottom&left&right
```



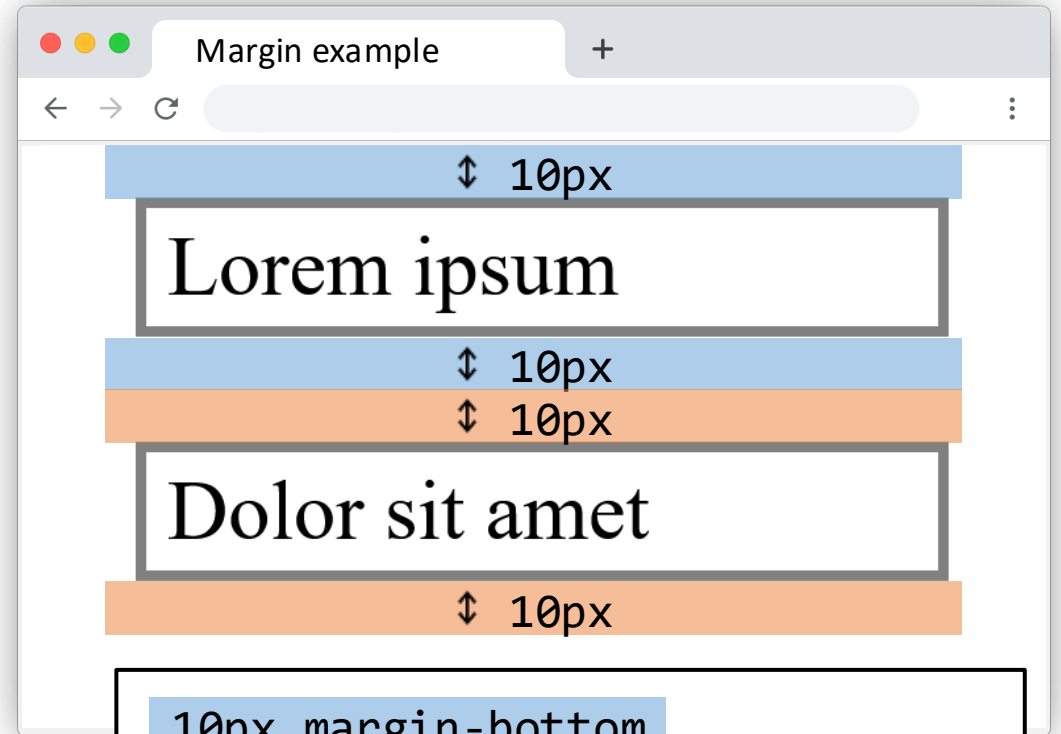
Margin

Why doesn't this ...



```
div {  
  border: 2px solid grey;  
  padding 2px 4px;  
  margin: 10px;  
}
```

... look more like this?



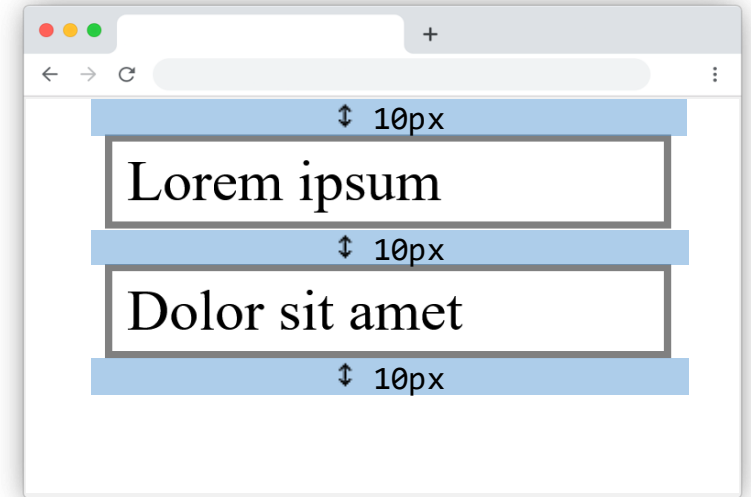
```
10px margin-bottom  
+  
10px margin-top  
= 20px?
```

Margin collapsing

Sometimes the top and bottom margins of block elements are combined ("collapsed") into a single margin.

Generally, [margin collapsing](#) applies when:

- Elements are siblings
- Elements are block-level (**not** inline-block)

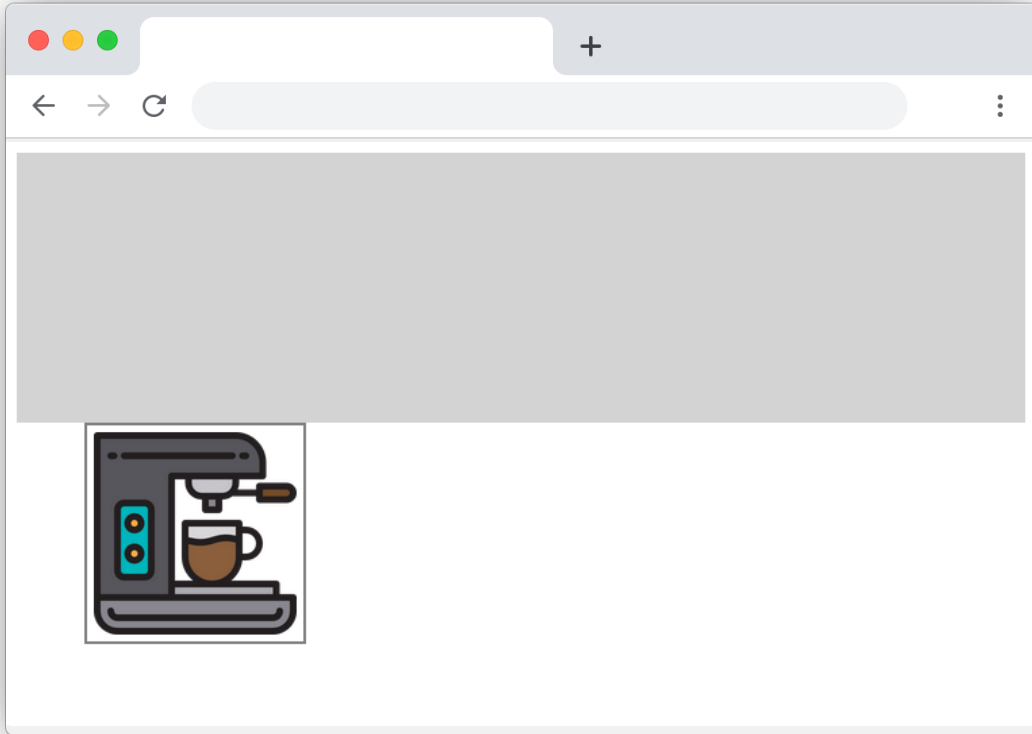


The collapsed margin is $\max(\text{above.margin-bottom}, \text{below.margin-top})$.

(There are [some exceptions](#) to this, but when in doubt, use the Page Inspector to see what's going on.)

Margin negative

Margin can be set to negative values.

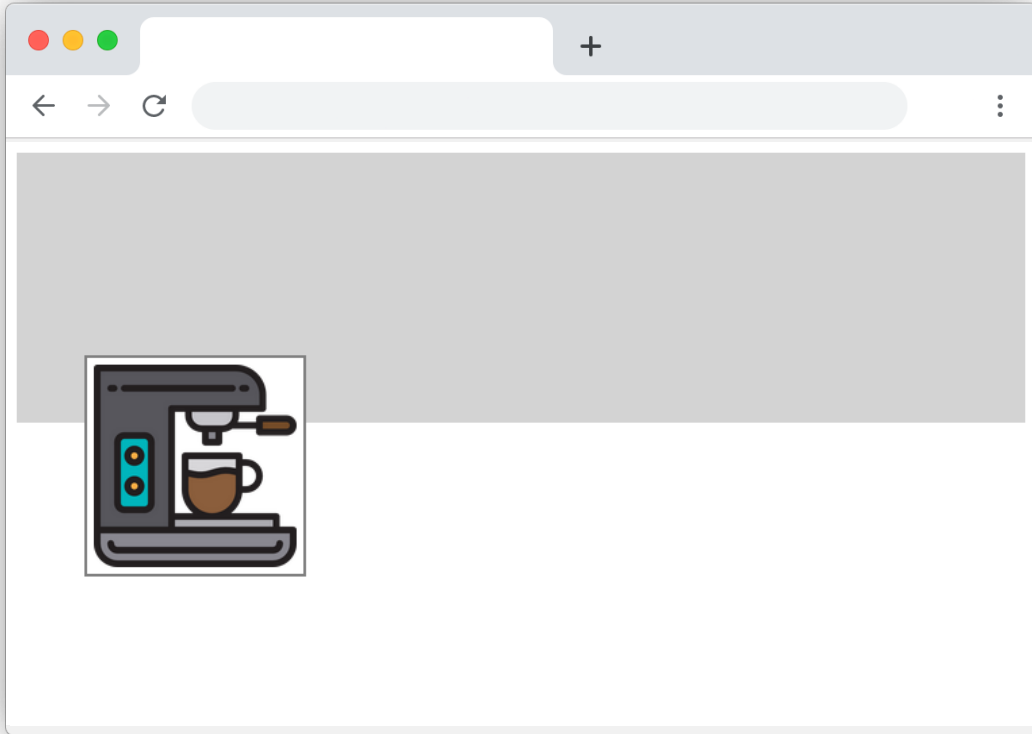


```
<div id="header"></div>
<div id="profile">
  
</div>
```

```
#header {
  background-color: lightgrey;
  height: 200px;
}
img {
  border: 2px solid grey;
  height: 150px;
  margin-left: 50px;
}
```

Margin negative

Margin can be set to negative values.

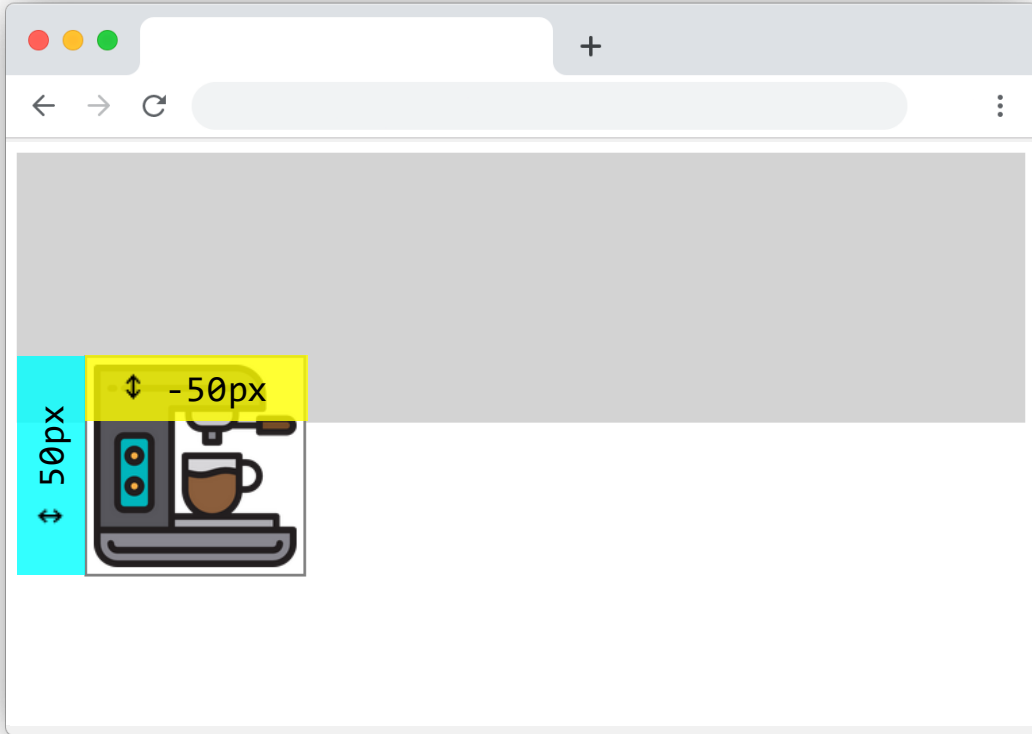


```
<div id="header"></div>
<div id="profile">
  
</div>
```

```
#header {
  background-color: lightgrey;
  height: 200px;
}
img {
  border: 2px solid grey;
  height: 150px;
  margin-left: 50px;
  margin-top: -50px;
}
```

Margin negative

Margin can be set to negative values.

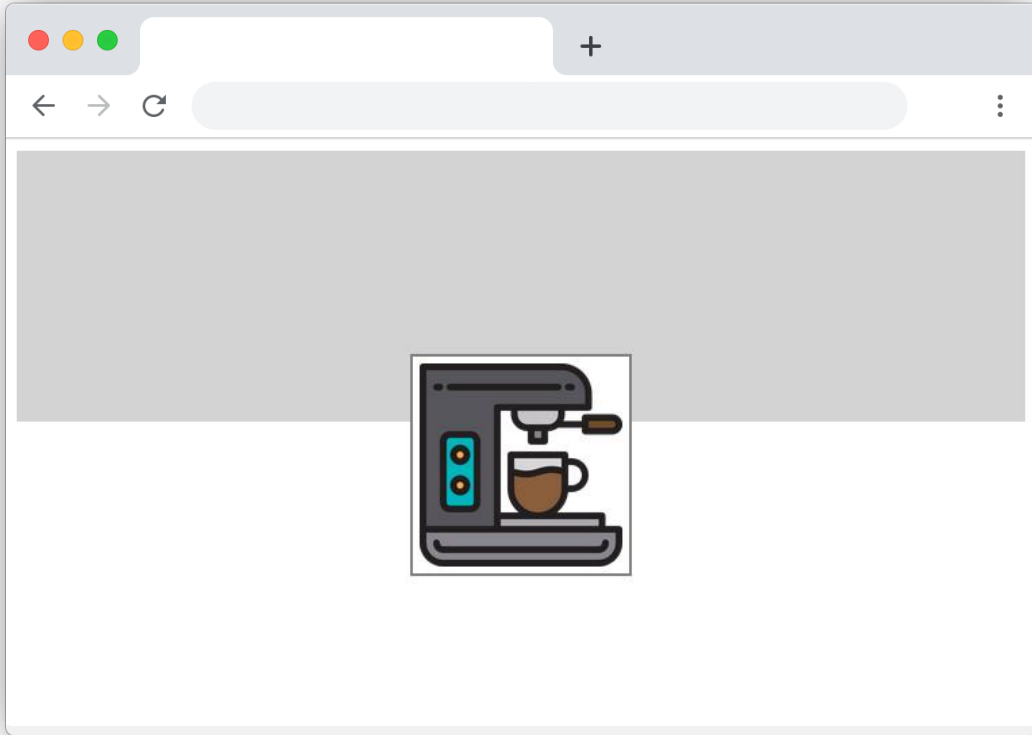


```
<div id="header"></div>
<div id="profile">
  
</div>
```

```
#header {
  background-color: lightgrey;
  height: 200px;
}
img {
  border: 2px solid grey;
  height: 150px;
  margin-left: 50px;
  margin-top: -50px;
}
```

Margin auto

Margin can be set to auto.



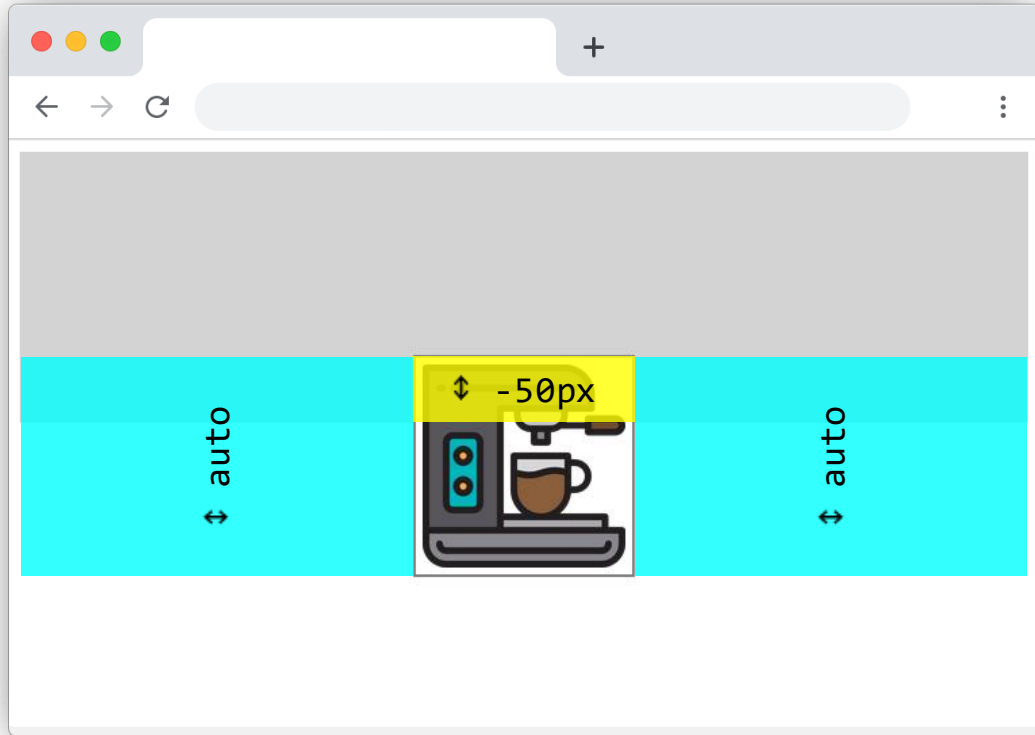
The browser selects a suitable margin to use.
Can be used to centre block elements.

```
<div id="header"></div>
<div id="profile">
  
</div>
```

```
#header {
  background-color: lightgrey;
  height: 200px;
}
img {
  border: 2px solid grey;
  height: 150px;
  display: block;
  margin-left: auto;
  margin-right: auto;
  margin-top: -50px;
}
```

Margin auto

Margin can be set to auto.



The browser selects a suitable margin to use.
Can be used to centre block elements.

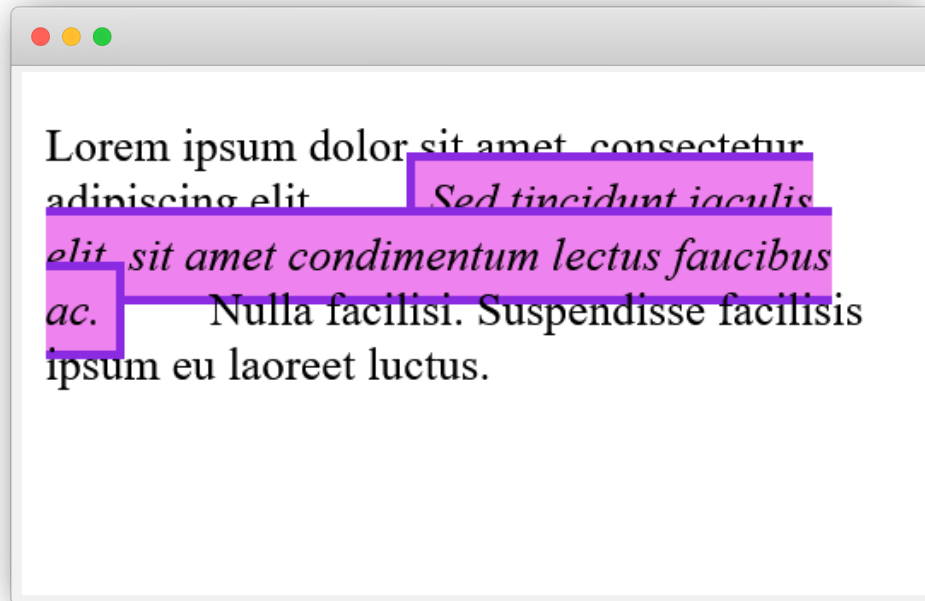
```
<div id="header"></div>
<div id="profile">
  
</div>
```

```
#header {
  background-color: lightgrey;
  height: 200px;
}
img {
  border: 2px solid grey;
  height: 150px;
  display: block;
  margin-left: auto;
  margin-right: auto;
  margin-top: -50px;
}
```

Box model for inline elements

Similar to block elements, but the box is different

- wraps around, as expected from text
- padding and border work the same
- margin-left and margin-right work the same
- margin-top and margin-bottom do not apply



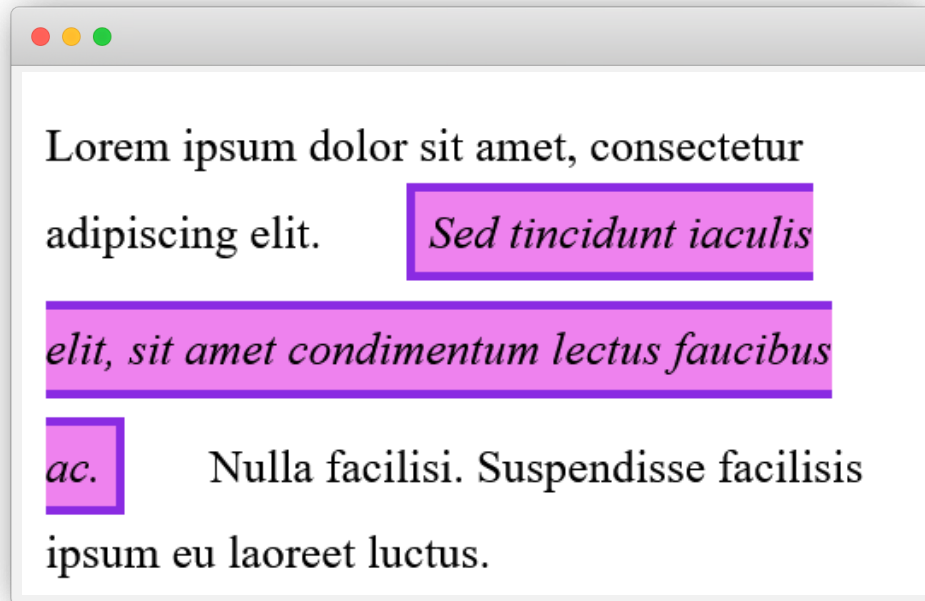
```
<p>
  Lorem ipsum ...
  <em>Sed tincidunt ...</em>
  Nulla ...
</p>
```

```
p {
  background-color: violet;
  border: 3px solid blueviolet;
  padding: 5px;
  margin: 25px;
}
```

Box model for inline elements

Similar to block elements, but the box is different

- wraps around, as expected from text
- padding and border work the same
- margin-left and margin-right work the same
- margin-top and margin-bottom do not apply
- line-height controls line height and spacing



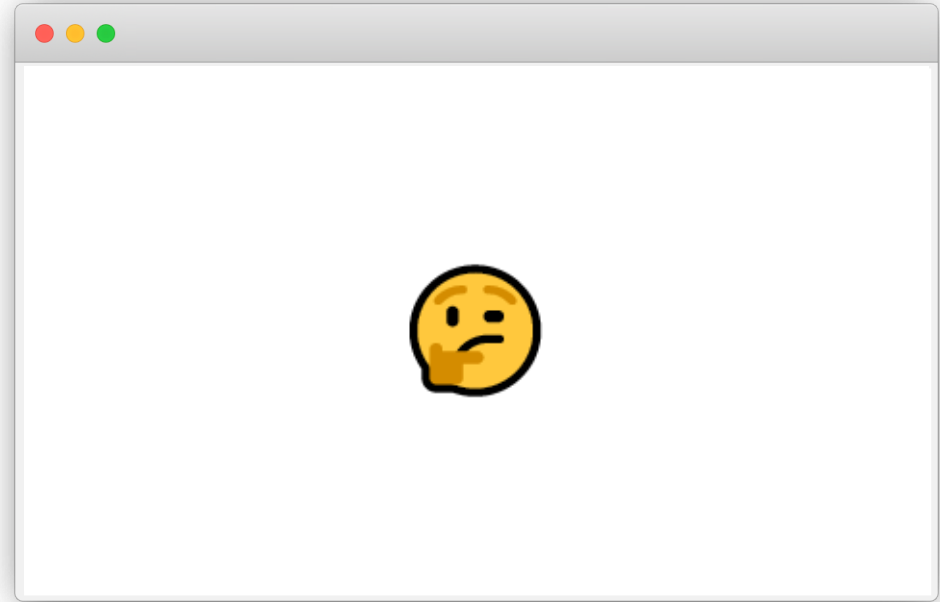
```
<p>
  Lorem ipsum ...
  <em>Sed tincidunt ...</em>
  Nulla ...
</p>
```

```
p {
  background-color: violet;
  border: 3px solid blueviolet;
  padding: 5px;
  margin: 25px;
  line-height: 40px;
}
```

What does this look like?

```
<div>  
  <p>Lorem ipsum</p>  
  <p>Dolor sit amet</p>  
</div>
```

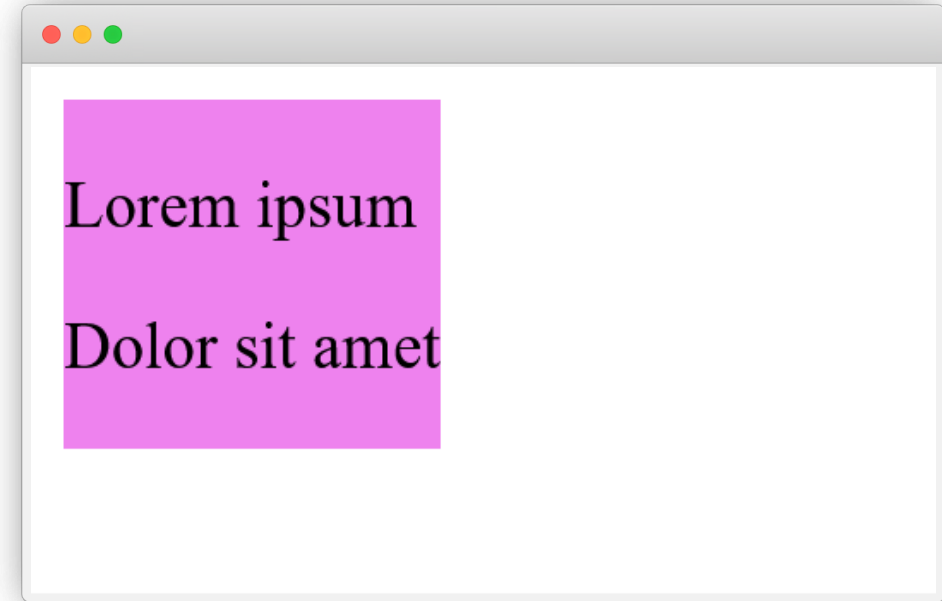
```
div {  
  display: inline-block;  
  background-color: violet;  
}
```



What does this look like?

```
<div>  
  <p>Lorem ipsum</p>  
  <p>Dolor sit amet</p>  
</div>
```

```
div {  
  display: inline-block;  
  background-color: violet;  
}
```

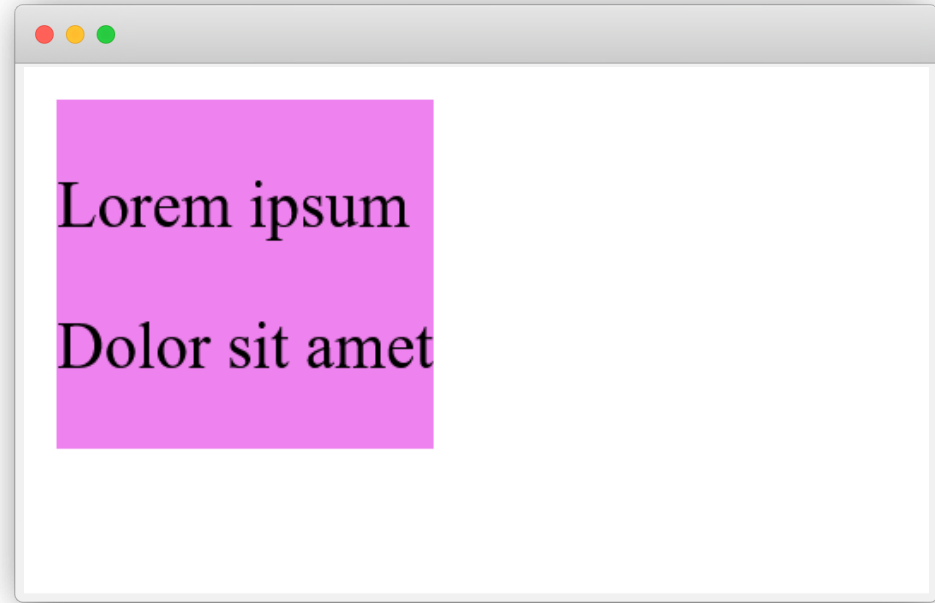


- Why is there a margin?

What does this look like?

```
<div>  
  <p>Lorem ipsum</p>  
  <p>Dolor sit amet</p>  
</div>
```

```
div {  
  display: inline-block;  
  background-color: violet;  
}
```

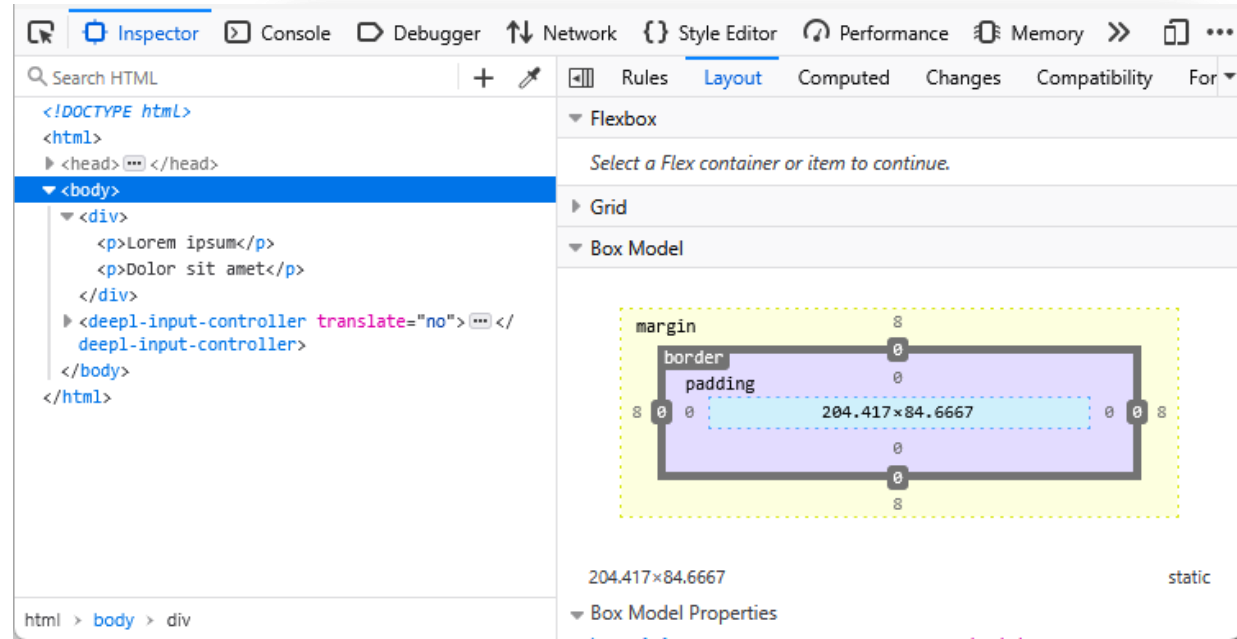


- Why is there a margin?
- Let's use the browser's page inspector to figure it out

What does this look like?

```
<div>
  <p>Lorem ipsum</p>
  <p>Dolor sit amet</p>
</div>
```

```
div {
  display: inline-block;
  background-color: violet;
}
```

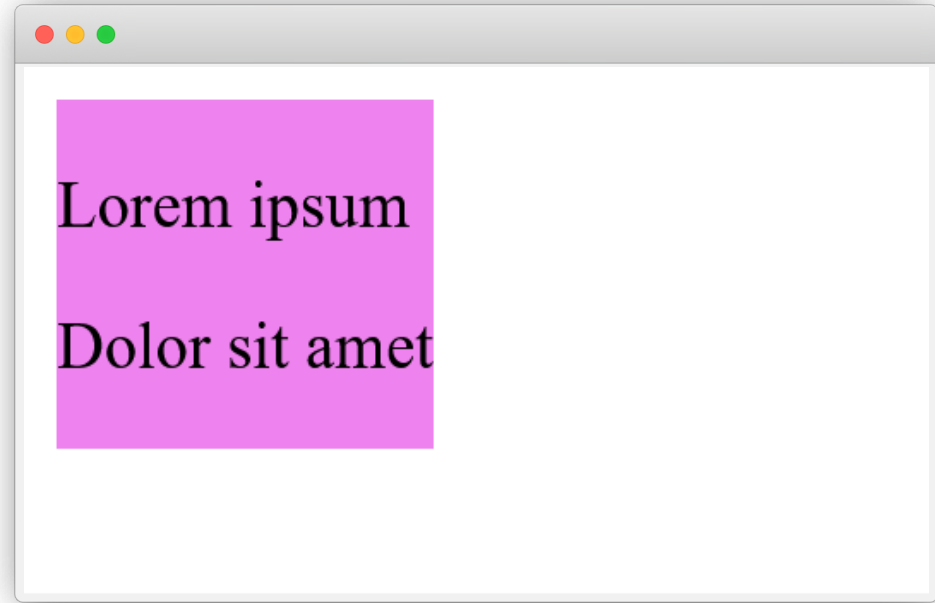


- Why is there a margin?
- Let's use the browser's page inspector to figure it out

What does this look like?

```
<div>  
  <p>Lorem ipsum</p>  
  <p>Dolor sit amet</p>  
</div>
```

```
div {  
  display: inline-block;  
  background-color: violet;  
}
```

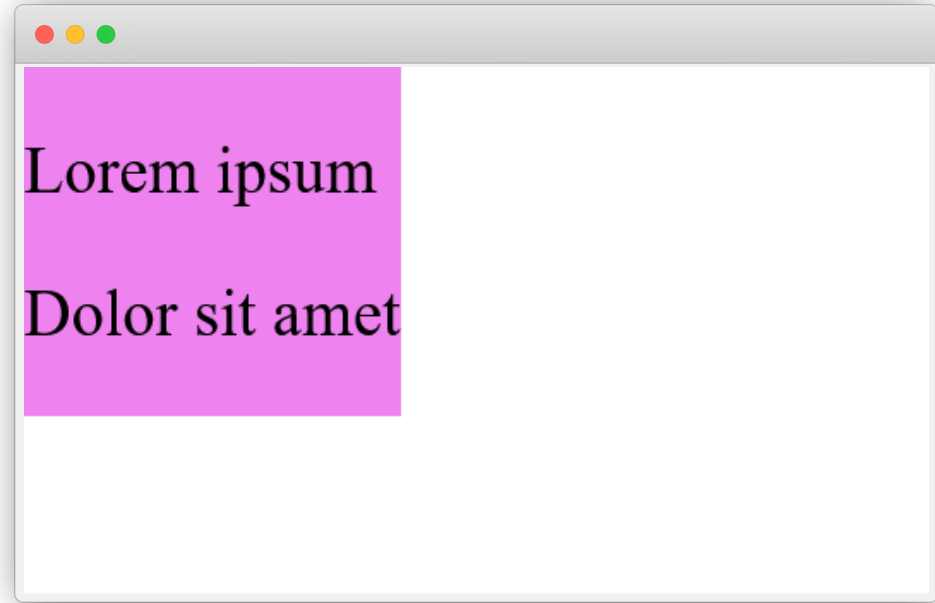


- Why is there a margin?
- Let's use the browser's page inspector to figure it out
- The body element has a default margin!

What does this look like?

```
<div>
  <p>Lorem ipsum</p>
  <p>Dolor sit amet</p>
</div>
```

```
div {
  display: inline-block;
  background-color: violet;
}
body {
  margin: 0px;
}
```



- Why is there a margin?
- Let's use the browser's page inspector to figure it out
- The body element has a default margin!

Review

Every element is composed of 4 layers:

- the element's content
- the **border** around the element's content
- **padding** space between the content and border (inside)
- a **margin** clears the area around border (outside)
 - Margin of sibling block elements collapses
 - Margin top and bottom are ignored for inline elements
 - Margin can also be negative or auto

